VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI– 590018, KARNATAKA, INDIA

A PROJECT REPORT

on

"Swaraaha: Stutter Classification and Localization System"

Submitted in partial fulfilment of the requirements for the award of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Submitted By

Name	USN
K SHREEKRISHNA UPADHYAYA	4VP23AI020
M CHETHAN KESHAV BHAT	4VP23AI023
SKANDA PRASAD K	4VP23AI051
SRINIVAS HEGDE M	4VP23AI054

Under the Guidance of

Prof. Ajay Shastry C G

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt., of Karnataka

Nehru Nagar, Puttur - 574 203, DK, Karnataka, India.

November, 2026

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]

Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka

Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

Certified that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** is carried out by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020**, **4VP23AI023, 4VP23AI051 and 4VP23AI054** respectively bonafide students of **Vivekananda College of Engineering & Technology, Puttur** in partial fulfilment for the award of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** of the **Visvesvaraya Technological University**, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated during Internal Assessment have been incorporated in the report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide	Signature of the HOD	Signature of the Principal
Prof. Ajay Shastry C G	**Dr. Radhika Shetty D S**	**Dr. Purushothama Chippar**

EXTERNAL VIVA

Name of the Examiners	Signature with date
1…….....	.....
2…….....	.....

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha Puttur (R)]
Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka
Nehru Nagar, Puttur - 574203, DK, Karnataka, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that **"Swaraaha: Stutter Classification and Localization System"** submitted by **Mr. K Shreekrishna Upadhyaya, Mr. M Chethan Keshav Bhat, Mr. Skanda Prasad K, Mr. Srinivas Hegde M** bearing USNs **4VP23AI020, 4VP23AI023, 4VP23AI051 and 4VP23AI054** to the Visvesvaraya Technological University, Belagavi, in partial fulfillment for the award of the degree of **Bachelor of Engineering** in **Artificial Intelligence & Machine Learning** is a *bonafide* record of project work carried out by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

_____

Signature of the Guide
Prof. Ajay Shastry C G
Department of AI & ML
VCET Puttur

DECLARATION

We, **Mr. K Shreekrishna Upadhyaya (4VP23AI020), Mr. M Chethan Keshav Bhat (4VP23AI023), Mr. Skanda Prasad K (4VP23AI051), Mr. Srinivas Hegde M (4VP23AI054)** students of B.E. 7th Semester in Artificial Intelligence & Machine Learning, **Vivekananda College of Engineering & Technology**, Puttur, hereby declare that the project work entitled **"Swaraaha: Stutter Classification and Localization System"** has been carried out by us at VCET, Puttur, under the guidance of **Prof. Ajay Shastry C G,** Assistant Professor, Department of Artificial Intelligence & Machine Learning, Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfilment of the requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence & Machine Learning** by **Visvesvaraya Technological University**, Belagavi during the academic year 2026-2027.

Name of the students	USN	Signature with date
K Shreekrishna Upadhyaya	4VP23AI020	
M Chethan Keshav Bhat	4VP23AI023	
Skanda Prasad K	4VP23AI051	
Srinivas Hegde M	4VP23AI054	

Date:

Place: Puttur

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide, **Prof. Ajay Shastry C G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S** for her guidance and inspiration.

We would like to thank our Principal, **Dr. Purushothama Chippar.** for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses. Roughly 1% of people worldwide are affected, and the condition can reach into their social, professional, and personal life. Diagnosis today still relies mostly on manual assessment by speech-language pathologists, which is subjective, time-consuming, and not available to everyone. This project builds an automated system that uses deep learning to detect, classify, and analyze stuttering events in speech recordings. The system identifies the types of dysfluencies present and reports when they occur. Users record or upload audio through a simple interface and receive the results in an accessible format. The result is a tool for early stutter detection and objective speech evaluation, useful to both speech-language pathologists and people in treatment. The system also contributes to the growing use of artificial intelligence in healthcare, supporting diagnosis, therapy, and an improved quality of life for people who stutter.

TABLE OF CONTENTS

Title	Page No.
LIST OF TABLES	I
LIST OF FIGURES	II
LIST OF ABBREVIATIONS	III
CHAPTER 1 INTRODUCTION	**01**
1.1 INTRODUCTION TO THE PROJECT	01
1.2 PROPOSED METHOD	01
1.3 LITERATURE SURVEY	02
1.4 PROBLEM STATEMENT	12
1.5 OBJECTIVES	12
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	**14**
2.1 INTRODUCTION	14
2.2 FUNCTIONAL REQUIREMENTS	14
2.2.1 User Requirements	14
2.2.2 System Requirements	15
2.3 NON-FUNCTIONAL REQUIREMENTS	15
2.3.1 Performance	15
2.3.2 Scalability	16
2.3.3 Availability	16
2.3.4 Usability	16
2.4 Software Requirements	16
2.5 Hardware Requirements	17
CHAPTER 3 SYSTEM DESIGN	**18**
3.1 INTRODUCTION	18
3.2 SYSTEM ARCHITECTURE AND DESIGN	18
3.3 FLOWCHART	21
3.4 USE CASE DIAGRAM AND DESCRIPTIONS	23
3.5 WORKFLOW DIAGRAM	25
3.6 SEQUENCE DIAGRAM	26
CHAPTER 4 SYSTEM IMPLEMENTATION	**28**
4.1 INTRODUCTION	28
4.2 ALGORITHM	28
4.3 IMPLEMENTATION REQUIREMENTS	29
4.3.1 Train the Model	30

4.3.2 Test the Model 30
4.3.3 Model Evaluation 31
4.4 USER INTERFACE 32
CHAPTER 5 SYSTEM TESTING 34
5.1 INTRODUCTION 34
5.2 TYPES OF TESTS 34
5.2.1 Unit Testing 34
5.2.2 Integration Testing 35
5.2.3 Functional Testing 35
5.2.4 System Testing 35
5.3 RESULT AND ANALYSIS 36
5.3.1 Model Evaluation of the Wav2Vec2 Classifiers 36
5.3.2 Wav2Vec2 Classifier Training 40
5.3.3 Model Evaluation of the CNN-based Models 41
5.3.4 CNN-based Model Training 42
5.3.5 Hyper Parameter Tuning 42
5.3.6 Performance Analysis of the Models 43
CHAPTER 6 SCREENSHOTS 45
CHAPTER 7 CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS 48
7.1 CONCLUSION 48
7.2 SCOPE FOR FUTURE ENHANCEMENTS 49
REFERENCES 50
PERSONAL PROFILE 54

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Literature Survey	07
5.1	Multitask Wav2Vec2 classifier results	36
5.2	Wav2Vec2 localizer evaluation results	39
5.3	Threshold optimization results	42
5.4	Comparative study of seven architecture variants	43

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System Architecture	19
3.2	Flowchart	22
3.3	Use Case Diagram	24
3.4	Workflow Diagram	25
3.5	Sequence Diagram	26
5.1	Per-class F1 of the multitask Wav2Vec2 classifier	37
5.2	Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier	37
5.3	Confusion matrices for the five binary classifiers	38
5.4	Frame-level metrics of the Wav2Vec2 localizer	40
5.5	Per-class tuned F1 on the Boli set	43
6.1	Home page of the web interface	45
6.2	Results page of the web interface	45
6.3	Home page of the desktop application	46
6.4	Analysis result page of the desktop application	47

LIST OF ABBREVIATIONS

Abbreviation	Description
ASR	Automatic Speech Recognition
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPRC	Area Under the Precision-Recall Curve
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HMM	Hidden Markov Model
HuBERT	Hidden-Unit BERT
isWER	Intended Speech Word Error Rate
LLM	Large Language Model
LPC	Linear Predictive Coding
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
PCA	Principal Component Analysis
PWS	People Who Stutter
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SEP-28K	Stuttering Event Detection Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State of the Art
SVM	Support Vector Machine
TTS	Text-to-Speech
UAR	Unweighted Average Recall
UI	User Interface
VITS	Variational Inference with Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Language Model
WER	Word Error Rate
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE PROJECT

Speech fluency lets a person put thoughts and feelings into words. Stuttering breaks that flow through involuntary repetitions, prolongations, or blocks. It affects people at every age and can wear down confidence, which makes conversation and social contact more difficult. Spotting the patterns early helps therapy and day-to-day management.

Manual detection and assessment still depend heavily on a speech-language pathologist who listens to recordings and marks what they hear. Two clinicians can disagree about the same sample, the work is slow, and repeating it tires them. That reliance on human listening also makes large-scale assessment and continuous monitoring hard to organise. Deep learning and speech signal processing now offer a scalable alternative. This project builds a system that detects, classifies, and analyzes five stuttering types: prolongations, blocks, sound repetitions, word repetitions, and interjections.

The pipeline preprocesses input audio, turns it into embeddings, and analyzes it with Wav2Vec 2.0, which captures fine acoustic detail and dependencies across longer stretches of speech. A second module localizes the stuttering events and returns their timestamps. Those timestamps line up with the transcript, so the system flags only the affected segments. The finished model sits behind a simple interface where a user records or uploads a sample and gets timestamped detections, class labels, and confidence scores.

The aim is an accurate, objective, and accessible diagnostic tool for speech-language pathologists and people who stutter. The work also supports the use of artificial intelligence in healthcare, where the same analysis can directly aid diagnosis, therapy, and a better quality of life.

1.2 PROPOSED METHOD

The proposed method is a deep learning system that detects and classifies speech dysfluencies early, using audio recordings as input. Preprocessing removes noise, balances loudness, and sharpens acoustic detail across all samples. Segmentation isolates the regions that carry speech so silence and background noise drop out. Augmentation such as noise injection, time and pitch shifting, and amplitude scaling exposes the model to different speaking rates and recording conditions. The system then puts each input into one of five dysfluency types and marks where the event occurred, which supports early diagnosis and

treatment planning. Automating this work cuts manual effort, reduces observer bias, and makes the assessment more reliable.

1.3 LITERATURE SURVEY

P. Arbajian et al. [1] The paper studies how the choice of speech segment affects stutter block detection. The authors extract acoustic features from annotated samples and compare classifiers under different segmentation setups, including several windowing approaches. Accuracy depends strongly on segment length: some lengths capture dysfluency patterns better, while poor segmentation drops temporal information and performance falls. The finding shaped the preprocessing stage of this project.

V. Mitra et al. [2] This study tunes an existing hybrid ASR system so a voice assistant handles dysfluent speech better. The aim is to cut recognition errors caused by repetitions and unintended insertions. The method changes decoding parameters: it raises the penalty for inserted words and lowers the weight of the acoustic model, so the language model filters repeated or disfluent segments and the recogniser focuses on meaningful speech. On recordings from 18 participants with different stuttering severity, intended speech word error rate fell by 24% relative. Parameter tuning alone achieved that gain without rebuilding the ASR architecture.

P. Mohapatra et al. [3] The paper introduces disfluencyNet, a model that detects speech disfluencies from contextual representations. Contextual embeddings feed a classification network that models dependencies in speech and catches repetitions and pauses. Data distillation reduces the need for a large training set. On SEP-28k and FluencyBank, the model matched baseline accuracy with a quarter of the data. That result supports the contextual embeddings and efficient training used in this project.

J. Liu et al. [4] This paper proposes a method for detecting disfluencies in multilingual speech of different lengths, using the contextual embeddings of wav2vec 2.0. A classification network takes the embeddings, and data distillation keeps only audio fragments where three human annotators agree on the label. Tests on multilingual datasets with varied speech lengths beat other approaches. The work supports the use of pretrained models and data filtering for robustness across speakers.

A. Romana et al. [5] The study detects disfluencies directly from untranscribed audio. A frame-level Bi-LSTM fusion model combines WavLM acoustic features with BERT representations taken from transcripts produced by a fine-tuned Whisper model. The com-

bination captures low-level speech patterns and higher-level context, and it tolerates ASR errors and misalignment. In noisy real-world conditions the multimodal setup was more robust and accurate than either modality alone.

D. Wagner et al. [6] This paper uses large language models to combine acoustic and linguistic representations for dysfluency detection. Acoustic features come from wav2vec 2.0 and transcriptions from Whisper; the two are fused into a joint input that an LLM classifies into stutter types, including repetitions, prolongations, and blocks. The acoustic-lexical combination beat models that used only audio or only text. That supports the plan in this project to combine audio and language signals.

S. A. Sheikh et al. [7] The study develops multi-contextual StutterNet, a framework that addresses class imbalance and limited data. A multi-branch architecture processes several contextual representations of the same speech. Class-balanced loss gives more weight to underrepresented stutter categories, and augmentation diversifies the training set. Detection was more robust across speech conditions, especially for less frequent dysfluency types. This project borrows the same three ideas: augmentation, balanced loss, and multi-context learning.

X. Zhou et al. [8] The paper treats spectrograms as images and adapts a YOLO-style detector to find dysfluencies in the time-frequency domain. A region-based model learns frequency patterns and changes over time, and speech-text alignment links audio segments to their words for context. The model predicts both the dysfluency type and the time region where it occurs, with strong results across several types. This project follows that region-wise, spectrogram-based approach.

X. Zhou et al. [9] Stutter-Solver is an end-to-end, YOLO-inspired model for multilingual dysfluency detection. It treats speech spectrograms as image-like inputs and applies a region-based detector. To cope with scarce data, the authors generated synthetic datasets (VCTK-Pro, VCTK-Art, and AISHELL3-Pro) with articulatory and text-to-speech simulations. The model performed well across datasets and languages, which supports the multilingual aims and synthetic data plans in this project.

J. Zhang et al. [10] This paper proposes LLM-Dys, a method for generating large dysfluent speech datasets with large language models. An LLM simulates realistic dysfluency patterns and writes labelled dysfluent text, which VITS converts into audio. Compared with rule-based generation, the speech keeps more natural prosody and contextual vari-

ation. The dataset covers 11 dysfluency categories at word and phoneme level, which allows fine-grained training and analysis. Models trained on it performed better on detection tasks.

T. Kourkounakis et al. [11] FluentNet is a CNN-LSTM architecture for end-to-end speech disfluency detection. CNN layers extract short-term spectral features from mel-spectrograms, and LSTM layers model temporal continuity, which suits recurring stutter patterns. On SEP-28k it reached 91.4% classification accuracy. Skipping handcrafted features reduced bias. The architecture informs the CNN models compared in this project.

V. Uloza et al. [12] This study develops an artificial intelligence-based framework for assessing substitution voicing in patients who have undergone laryngeal cancer surgery. The model uses a hybrid convolutional neural network architecture to analyse Mel-frequency spectrograms and cochleagrams extracted from speech recordings. Trained on 309 speech recordings and evaluated on 70 previously unseen samples, the system achieved an overall accuracy of 92.14%. The proposed Acoustic Substitution Voicing Index (ASVI) demonstrated a strong correlation of 0.863 with assessments performed by trained medical professionals. The lightweight architecture achieved a processing time of 3.56 ms, making it suitable for real-time speech analysis on CPU-based devices.

N. Zhang et al. [13] This meta-analysis of fMRI and EEG studies looks at the neural bases of speech production in people who stutter. The inferior frontal gyrus and basal ganglia show atypical activation during speech. Those findings give a biological grounding to acoustic detection, since the irregularities a model picks up reflect differences in neural control. The study proposes no computational model, but it helps explain what the detected events represent.

A. Baevski et al. [14] wav2vec 2.0 learns speech representations directly from raw audio in a self-supervised way. A convolutional encoder extracts latent features and a transformer adds context. Pretrained on 960 hours of unlabelled LibriSpeech and fine-tuned with a CTC loss, it performed strongly on phoneme recognition, speaker identification, and emotion recognition. It is the backbone of this project, and its pretraining is what makes feature extraction possible without a large labelled stutter corpus.

P. Khanna et al. [15] StuD is a multimodal stuttering detector that combines Wav2Vec 2.0 and HuBERT acoustic features with Llama-2 linguistic features and retrieval-augmented generation. A fusion strategy weights the acoustic and linguistic embeddings by input

quality, and the retrieval component pulls similar historical cases to classify rare patterns. On SEP-28k and FluencyBank it performed strongly across event types. The fusion design supports the combined audio and text analysis in this project.

C. Lea et al. [16] SEP-28K contains over 28,000 clips from stuttering support group podcasts, annotated for blocks, prolongations, repetitions, interjections, and revisions. Clips carry crowdsourced labels from non-expert listeners and expert labels from speech-language pathologists. Scaling annotation from 10k to 28k clips improved F1 by 28% for blocks and 24% for prolongations. It is the main benchmark for this project.

O. Shonibare et al. [17] This paper proposes a two-stage approach to ASR on stuttered speech with limited labelled data. A context-aware classifier trained on a small amount of labelled stuttering data finds dysfluency regions, and a modified ASR decoder passes over them so transcription focuses on fluent segments. Word error rate fell by 12% to 71% across severity levels. The detect-then-transcribe design shows that detection and speech processing can be separated, which is how this project's pipeline is organised.

S. P. Bayerl et al. [18] The study applies a fine-tuned wav2vec 2.0 to dysfluency detection in clinical therapy recordings. Acoustic embeddings go through a multi-task framework and an SVM classifier that separates dysfluency types. On FluencyBank and the German KSoF dataset, F1 improved by 27% over baselines. Predicting presence and type together lets therapists track progress across sessions, which shows that pretrained speech models transfer to clinical assessment.

S. P. Bayerl et al. [19] This paper treats stuttering detection as a multi-label problem, since dysfluencies often occur together. A modified wav2vec 2.0 assigns an independent probability to each type, so overlapping events are detected at once. It performed well on SEP-28k-Extended and transferred from English to German. The formulation matches clinical reality: one utterance can contain a block, a prolongation, and repetitions together.

R. Gong et al. [20] AS-70 is the first large Mandarin stuttered speech dataset, with 70 hours from speakers at different severity levels. It covers age groups, genders, and stuttering types, and includes word- and syllable-level dysfluency labels. The corpus is open source. It extends stuttering data beyond English and makes multilingual systems possible.

X. Liu et al. [21] This paper combines Conformer blocks for local acoustic patterns with BiLSTM layers for long-range temporal dependencies. The multi-task framework predicts dysfluency type and severity together. It took first place in the SLT 2024 Stuttering Detection Challenge and improved F1 by 39.8% on AS-70. Conformer's mix of convolution and self-attention captures local and global speech patterns, which suits the varied timing of stuttering events.

A. Batra et al. [22] Boli is a multilingual Indian language dataset with read and spontaneous speech from people who stutter. It covers five stuttering types, records demographic metadata, and includes self-reported severity ratings. The corpus fills a gap in non-English stuttering data, especially for South Asian languages, where prevalence and manifestation differ from Western populations.

A. Valente et al. [23] This study adds detailed clinical annotations from expert speech-language pathologists to FluencyBank. The labels cover audiovisual cues and secondary behaviours such as facial tension and eye blinking, plus physiological tension indicators, beyond the usual acoustic labels. The multi-dimensional scheme captures more of the clinical picture of severity. Such annotations allow models to account for the full stuttering experience rather than acoustic events alone.

A. Batra et al. [24] Boli is a multilingual Indian language dataset with read and spontaneous speech from people who stutter. It covers five stuttering types, records demographic metadata, and includes self-reported severity ratings. The corpus fills a gap in non-English stuttering data, especially for South Asian languages, where prevalence and manifestation differ from Western populations.

A. Valente et al. [25] This study adds detailed clinical annotations from expert speech-language pathologists to FluencyBank. The labels cover audiovisual cues and secondary behaviours such as facial tension and eye blinking, plus physiological tension indicators, beyond the usual acoustic labels. The multi-dimensional scheme captures more of the clinical picture of severity. Such annotations allow models to account for the full stuttering experience rather than acoustic events alone.

A. Romana et al. [26] The study releases an updated FluencyBank with word-level timestamps and refined disfluency annotations. Transcripts carry granular labels and word timing for each clip, so models can be measured on disfluent input. The resource supports

word-level localization, which is what this project needs to map events to the words where they occur.

X. Tang et al. [27] This paper presents annotation guidelines developed with people who stutter, standardising how stuttering events are labelled. The guidelines resolve ambiguous cases with categorical definitions and decision trees, and they cover annotator training, inter-annotator agreement, and edge cases. Community-informed guidelines raise annotation quality and reproducibility, which helps dataset construction and model evaluation.

A. Romana et al. [28] The study releases an updated FluencyBank with word-level timestamps and refined disfluency annotations. Transcripts carry granular labels and word timing for each clip, so models can be measured on disfluent input. The resource supports word-level localization, which is what this project needs to map events to the words where they occur. Portuguese stuttering differs from English in syllable structure and in how dysfluencies surface, and the method addresses those patterns. It extends detection to an underrepresented language.

The studies cover acoustic classifiers, self-supervised speech representations, multimodal fusion, and synthetic data. Table 1.1 compares them by features, strengths, and limitations. The field has moved from hand-crafted acoustic features toward large pretrained models. Multilingual coverage, localization, and clinical reliability remain open.

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al. [1]	Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation	Analyzed different speech segment lengths and sample selection methods for accurate stutter block detection using acoustic classifiers.	Improves detection precision through better segment selection.	Sensitive to segmentation configuration.
2	Vikramjit Mitra et al. [2]	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoustic model influence to sup-	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.

Sl. No	Author	Title	Features	Pros	Cons
			press repetition errors		
3	Payal Mohapatra et al. [3]	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples for efficient low-resource training	Achieves strong results with reduced training data	Depends on carefully filtered annotations
4	Jiajun Liu et al. [4]	Automatic Speech Disfluency Detection Using wav2vec 2.0 for Different Languages with Variable Lengths	Applied wav2vec 2.0 contextual embeddings for multilingual disfluency detection across variable speech durations.	Supports multilingual detection with strong contextual learning.	High resource requirements and limited availability of multilingual stuttering datasets.
5	Amrit Romana et al. [5]	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al. [6]	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requires heavy computational resources.
7	Shakeel A. Sheikh et al. [7]	Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning	Proposed multi-contextual StutterNet with augmentation and weighted loss for robust stuttering detection.	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.
8	X. Zhou et al. [8]	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stutter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.

Sl. No	Author	Title	Features	Pros	Cons
9	Xuanru Zhou et al. [9]	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Developed multilingual YOLO-based dysfluency detector using synthetic articulatory and TTS-generated datasets.	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
10	Jinning Zhang et al. [10]	Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection	Proposed LLM-Dys framework using LLM-generated dysfluent text and VITS TTS for scalable corpus creation.	Generates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.
11	T. Kourkounakis et al. [11]	FluentNet: End-to-End Detection of Stuttered Speech Disfluencies with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependencies in speech signals using SEP-28k dataset.	High accuracy in multi-class dysfluency detection.	Model complexity increases training time and requires GPU-based systems.
12	V. Uloza et al. [12]	An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing	Uses a CNN-based hybrid deep learning model with Mel-frequency spectrograms and cochleagrams to assess substitution voicing.	Achieves 92.14% accuracy with a lightweight architecture and 3.56 ms processing time.	Trained on Lithuanian male speech recordings, limiting generalization to other languages and populations.
13	N. Zhang et al. [13]	Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis	Analyzed fMRI/EEG studies to identify brain regions linked to speech dysfluency.	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.
14	A. Baevski et al. [14]	wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations	Self-supervised pre-training of speech representations using masked prediction with transformer and convolutional encoder on raw audio.	Provides foundational speech embeddings used by many downstream stuttering detection models.	Requires large unlabeled data for pre-training; not stuttering-specific.
15	P. Khanna et al. [15]	StuD: A Multimodal Approach for Stuttering Detection	Combined Wav2Vec 2.0, HuBERT acoustic features with Llama-2 lin-	Achieves SOTA on SEP-28k and FluencyBank with adaptive retrieval.	Heavy computational requirements for LLM and RAG components.

Sl. No	Author	Title	Features	Pros	Cons
		with RAG and Fusion Strategies	guistic features and RAG-based adaptive classification.		
16	C. Lea et al. [16]	SEP-28k: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter	Large-scale dataset with 28k+ clips annotated for 5 stuttering event types from podcast recordings.	Enables large-scale training with 28%/24% F1 gains from data scaling.	Podcast speech may not generalize to clinical or spontaneous settings.
17	O. Shonibare et al. [17]	Enhancing ASR for Stuttered Speech with Limited Data Using Detect and Pass	Two-stage approach: context-aware dysfluency detection followed by ASR that passes over detected events.	Achieves 12-71% WER reduction with minimal labeled data.	Relies on accurate first-stage detection; cascaded errors possible.
18	S. P. Bayerl et al. [18]	Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0	Fine-tuned wav2vec 2.0 with multi-task learning and SVM for therapy speech analysis.	27% F1 improvement on clinical therapy recordings.	Limited to therapy contexts; may not generalize to casual speech.
19	S. P. Bayerl et al. [19]	A Stutter Seldom Comes Alone - Cross-Corpus Stuttering Detection as a Multi-label Problem	Modified wav2vec 2.0 for simultaneous multi-label detection of co-occurring dysfluencies.	SOTA on SEP-28k-Extended with cross-language generalization.	Multi-label training increases model complexity and annotation requirements.
20	R. Gong et al. [20]	AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition and Stuttering Event Detection	First large-scale Mandarin stuttered speech dataset with verbatim transcriptions and detailed annotations.	Enables Mandarin stuttering research with 70h of diverse speaker data.	Limited to Mandarin; annotation process labor-intensive.
21	X. Liu et al. [21]	An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM	Conformer blocks with BiLSTM temporal modeling for multi-task dysfluency and severity prediction.	1st place SLT 2024 Challenge; 39.8% F1 improvement on AS-70.	Conformer architecture requires significant GPU memory and training time.
22	A. Batra et al. [22]	Boli: A Dataset for Understanding Stuttering Experience and Analyzing Stuttered Speech	Multi-lingual Indian language dataset with read and spontaneous speech across 5 stutter types.	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.

Sl. No	Author	Title	Features	Pros	Cons
23	A. Valente et al. [23]	Clinical Annotations for Automatic Stuttering Severity Assessment	Enhanced FluencyBank with expert clinician annotations including audiovisual and behavioral cues.	Provides clinical-grade multi-dimensional annotations for severity assessment.	Expert annotation is expensive and time-consuming to produce.
24	T. Grósz et al. [24]	Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering	Joint wav2vec2 framework for emotion recognition and stuttering detection from speech embeddings.	Demonstrates emotion-disfluency interconnection with 62.1% UAR.	Joint modeling may introduce task interference; limited to paralinguistic analysis.
25	X. Tang et al. [25]	Disability-First AI Dataset Annotation: Co-designing Stuttered Speech Annotation Guidelines with People Who Stutter	Standardized annotation protocols developed collaboratively with PWS for consistent dysfluency labeling.	Improves annotation quality and reproducibility across stuttering research.	Guidelines adoption requires community-wide coordination and training.
26	A. Romana et al. [26]	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition	Updated FluencyBank with word-level timestamps and refined disfluency annotations for precise temporal alignment.	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.
27	A. Batra et al. [27]	Boli: A Dataset for Understanding Stuttering Experience and Analyzing Stuttered Speech	Multi-lingual Indian language dataset with read and spontaneous speech across 5 stutter types.	Captures real-world Indian stuttering patterns across multiple languages.	Limited annotations; may need expert review for clinical applications.
28	A. Romana et al. [28]	FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition	Updated FluencyBank with word-level timestamps and refined disfluency annotations for precise temporal alignment.	Enables word-level dysfluency localization and more accurate model evaluation.	Limited to English stuttering patterns; requires updated annotation protocols.

Table 1.1: Summary of Literature Survey

1.4 PROBLEM STATEMENT

Stuttering assessment in clinical practice still relies on a speech-language pathologist who listens to recordings and marks dysfluencies by hand. The process is slow, subjective, and hard to reproduce, and disagreement between raters makes a patient's progress difficult to track. Most automated tools stop at binary detection: they report that a recording contains stuttering but not which type occurred or exactly where in the audio it occurred, which limits their use for therapy planning.

This project closes that gap with a single pipeline that takes an unconstrained speech recording and produces a structured analysis: whether stuttering is present, which of the five dysfluency types are involved, and the exact time regions where each event occurs. The system must also work without a large annotated corpus from the target speaker and must run fast enough for interactive assessment instead of offline batch processing.

1.5 OBJECTIVES

The project is built around four objectives, and each one supplies one stage of the final pipeline.

1. **Speech preprocessing:** clean and standardise the input audio so every later model gets consistent data. This covers resampling to a fixed rate, DC offset removal, peak normalization, silence trimming, fixed-length padding, and augmentation such as noise injection and time or pitch shifting, which together limit the effect of varying recording conditions and speaker loudness.
2. **Stutter detection:** decide whether a sample contains dysfluent speech. The binary output controls the rest of the analysis, so the model is trained to stay reliable across different speakers, languages, and background noise, not merely on clean studio recordings alone.
3. **Stutter classification:** identify which dysfluency types are present among prolongation, block, sound repetition, word repetition, and interjection. Because these events often co-occur, the system decides each type independently and gives it a separate confidence score instead of forcing one mutually exclusive label.
4. **Stutter localization:** find the time regions where detected events occur and align them with the corresponding words in the transcript. Clinicians get usable timestamps, so they can go straight to the affected segment and compare the detected events with what was said.

The objectives are ordered so each stage builds on the one before it. Preprocessing yields a normalized waveform and spectrogram for detection and classification. Detection narrows the analysis to samples that contain dysfluencies. Type-level scores come from classification, and localization maps events back onto the time axis and the transcript. Together these stages turn a recording into evidence that can be inspected at clip level and at event level. The classification scores summarise what was detected, while the localized intervals and aligned transcript show where it happened, which makes the result easier to review.

CHAPTER 2

REQUIREMENT SPECIFICATION AND ANALYSIS

2.1 INTRODUCTION

A deep learning framework for stuttering classification and localization must meet clinical and technical expectations at the same time. Before implementation, the project therefore fixes all its functional and non-functional requirements, the system components, and the workflow that connects them. The main goal is to replace manual speech-language assessment with a model that detects dysfluencies early and reports them objectively.

The system takes an audio recording, extracts features, and sorts it into the five stuttering types. It also locates each event in the signal. That should cut the time manual analysis takes and reduce variation between observers, without making the output hard to read. Wav2Vec 2.0 handles feature extraction, a Wav2Vec2 frame-level model handles localization, and Whisper produces the transcript. The system must run on standard hardware, cope with larger datasets as they arrive, offer a simple interface, and keep patient data secure and confidential.

2.2 FUNCTIONAL REQUIREMENTS

The functional requirements describe what the system must do to detect and classify stuttering events. Clinicians, speech-language pathologists, and people who stutter can upload a recording or record audio through the interface. The supported formats are WAV, MP3, FLAC, and M4A.

The system processes the audio and sorts it into five dysfluency types: prolongation, block, sound repetition, word repetition, and interjection. For each detected class, the interface shows the result with a confidence score, so the user can see how far to trust the prediction. The system also returns the timestamps of localized events. Waveforms, spectrograms, and overlays mark the regions that shaped the model's decision. Users can download the full analysis report, and uploaded audio and results are stored securely for later reference.

2.2.1 User Requirements

User requirements describe what all end users must be able to do. They reflect how clinicians, speech-language pathologists, researchers, and people who stutter will use the system while they assess or monitor fluency. Those users need an interface that is easy

to navigate, that shows classification results, visual interpretations, and confidence levels without technical complexity, and that works with little manual effort. For a speech analysis platform, the general requirements are:

- The user should be able to upload or record a speech sample for analysis.
- The user should be able to see classification results as soon as the analysis finishes.
- The system should provide detailed analysis reports.

2.2.2 System Requirements

System requirements cover the software and hardware capabilities the system needs. The stuttering detection system has to handle the computational load of deep learning speech analysis while delivering the operations described above. Hardware guidance appears later in this chapter.

These requirements include:

- The system must accept audio uploads securely and classify them.
- Users need a way to save previous analysis results and open them again.
- Adequate storage must be available for managing large speech datasets.

2.3 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements set the quality, performance, and usability standards the system has to meet. Performance comes first, because the models run deep learning and audio processing and must return results quickly. GPU acceleration and memory management keep the recordings moving through the pipeline. Reliability means consistent accuracy across datasets and fault tolerance when a run is interrupted. Scalability allows any larger datasets or extra dysfluency types without hurting overall performance. Usability means a simple interface for technical and non-technical users alike, with clear upload, analysis, and visualisation steps. Maintainability comes from modular code that is easy to update and retrain. Portability and availability keep the system running across platforms, with high uptime and regular backups.

2.3.1 Performance

- The system should process and analyse a speech sample within 3 to 5 seconds.
- The classification results should keep false negatives and false positives low.
- Performance metrics should be monitored regularly.

- The system should handle audio uploads without significant delay.

2.3.2 Scalability

- The system should handle growing speech datasets efficiently.
- It should scale across multiple clinics and research centres.
- The system should adapt to improvements in deep learning models.
- The system should handle a higher volume of concurrent users.

2.3.3 Availability

- The system should stay accessible at all times, with minimal downtime during clinical use.
- Components should tolerate faults so a single failure has less impact.
- The platform should keep serving users during updates or model improvements.

2.3.4 Usability

- The interface should be easy for clinicians, speech-language pathologists, and non-technical users.
- Audio upload and results should be easy to find and navigate.
- The visualisation outputs should help users understand the model's predictions for the analysed speech.

2.4 Software Requirements

The software requirements cover the tools used to develop, train, validate, and deploy the stutter detection models.

- **Operating System:** the system runs on Linux and Windows, including Windows 10 and above and common Linux distributions. The PySide6 desktop application and the web application run on either platform, and training and inference are supported on CPU or an NVIDIA GPU.
- **Deep Learning Framework:** PyTorch is the primary framework for building and training the models. It provides GPU support through torch.compile, mixed precision, and TensorFloat-32 (TF32), and it scales to the audio classification tasks in this project. Hugging Face Transformers supplies the pretrained speech models, Wav2Vec 2.0 for feature extraction and Whisper for timestamped transcription.

- **Libraries:** a range of Python libraries supports different stages of the project:
    - librosa: Used for audio preprocessing and manipulation, including loading, resampling, mel-spectrogram computation, and silence trimming.
    - NumPy: Provides support for numerical operations and array handling.
    - Pandas: managing structured data such as labels or metadata.
    - Scikit-learn: Used for data splitting, model evaluation metrics, and traditional machine learning algorithms.

2.5 Hardware Requirements

The hardware requirements give the minimum and recommended specifications for running the models.

- **Processor (CPU):** a multi-core processor such as an Intel Core i5 (8th generation or newer) or AMD Ryzen 5 and above. It provides enough power for audio preprocessing and training workflows.
- **Storage (Hard Disk):** at least 10 GB of free disk space for datasets, checkpoints, logs, and temporary files. An SSD is recommended for faster read and write speeds.
- **Memory (RAM):** at least 8 GB of RAM to run the training scripts and handle datasets, with 16 GB or more recommended during training and validation, when data loading and augmentation are active.
- **Graphics Processing Unit (GPU):** an NVIDIA GPU with CUDA support is recommended for faster training. Cards such as the RTX 2060 or higher (RTX 3060, RTX 3080) cut training time and handle larger datasets and deeper networks. CUDA and cuDNN must be installed for full compatibility with PyTorch.
- **Audio Input Device:** a working microphone is required for recording speech in the application.

The minimum configuration handles inference on short clips. Fine-tuning the larger pretrained backbones benefits from the recommended configuration, and a GPU with at least 8 GB of memory is preferable, though classification-only mode runs comfortably on a CPU.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

System design is a core stage in software development that sets out the overall structure and main functionality of a system. It turns user and technical requirements into a detailed framework that shows how different components, modules, and interfaces will interact. The process involves both high-level architecture, which lays out the system's major elements and their connections, and low-level design, which specifies detailed components such as data structures, algorithms, and communication flows.

System design aims to build a solution that meets user needs and still delivers strong performance, reliability, scalability, and security. It also values maintainability and flexibility, so the system can adapt to future changes and enhancements. Good system design needs close collaboration among developers, architects, and domain experts, so the architecture matches functional goals and business requirements. By organizing how each part of the system works and communicates, system design lays a solid foundation for efficient implementation and stable operation over time.

3.2 SYSTEM ARCHITECTURE AND DESIGN

The system architecture sets out the overall structure of the Stuttering Classification and Localization System, which uses deep learning. It gives a full picture of how the different components interact to process speech audio, extract useful features, and classify utterances as normal or dysfluent. A sound architecture keeps data flowing smoothly, stays modular and scalable, letting the model reach high diagnostic accuracy while remaining open to future changes. It covers all the main stages of the pipeline, from preprocessing and feature extraction to model training and classification, so that performance stays stable. It also supports smooth integration with user interfaces for audio upload, visualization, and prediction display. The design keeps data security and patient confidentiality central, so the analysis workflow stays ethical and reliable in practice. The architecture also leaves room for performance tuning, so stutter detection stays fast and accurate even on large speech datasets.

Figure 3.1 shows the proposed deep learning architecture for the automatic classification and localization of stuttering from speech audio. The framework combines two model pathways: five independent Wav2Vec 2.0 binary classifiers for classification with a

Wav2Vec2 frame-level pipeline for localization, which improves feature extraction and makes classification more reliable. The complete workflow is made up of five major phases: Input, Pre-processing, Feature Extraction, Classification, and Output. This dual-model structure learns richer representations and raises the overall detection accuracy. The architecture also separates reusable signal preparation from task-specific prediction. The same normalized waveform can go straight to classification, transcription, and localization, while the final fusion stage combines their outputs without repeating preprocessing work.

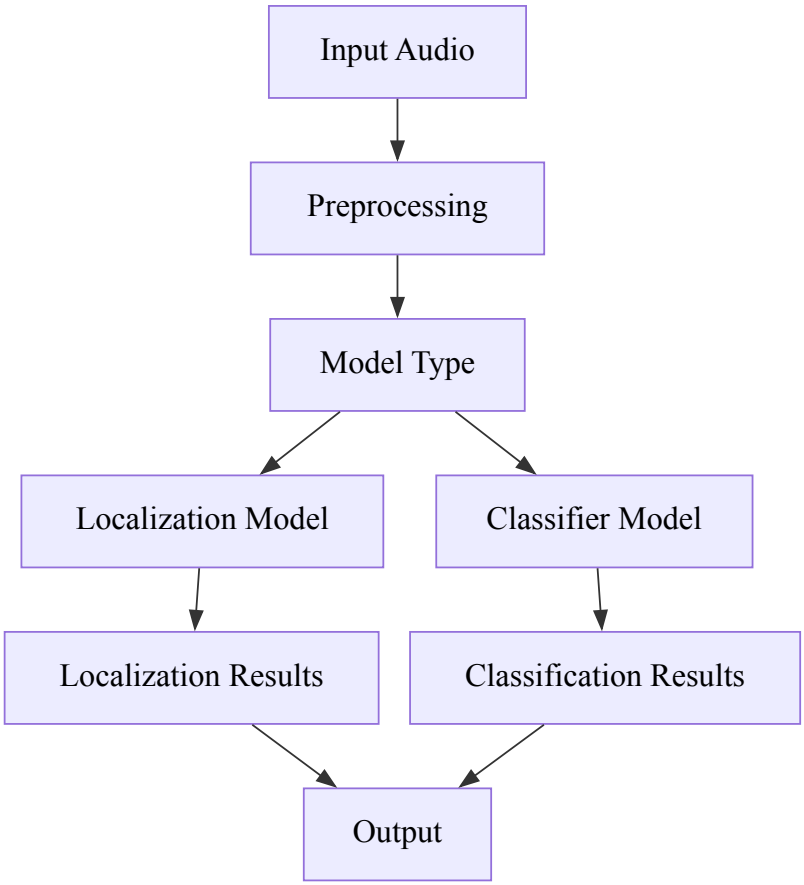


Figure 3.1: System Architecture

Speech Audio Input: The system starts by collecting the raw speech audio samples, which form the primary dataset for automatic stutter detection. Audio enters the system when the user records directly or uploads a pre-recorded file through the web interface or desktop application. The web app captures audio with MediaRecorder APIs; the desktop app uses sounddevice. Each sample may contain one or more dysfluencies, including blocks, prolongations, word repetitions, sound repetitions, and interjections. Recording quality matters a great deal, since differences in microphone hardware, background noise, and speaking rate can affect model reliability. The dataset includes recordings from Project Boli, SEP-28K, and UCLASS, and the ingestion pipeline resolves the English

material of all three sources. This stage gives the diagnostic system its clean data for the steps that follow.

Preprocessing: Before it is fed to the deep learning model, speech audio undergoes preprocessing to improve clarity, cut noise, and keep samples consistent. The audio is sent to the backend and converted to 16 kHz mono WAV with FFmpeg so every clip is handled the same way. The preprocessing pipeline also includes DC offset removal, peak normalization, and silence trimming to clean the signal and remove unwanted artifacts. Fixed-length padding truncates or pads each clip to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length. After this step the input audio is standardized and optimized, so the deep learning architecture can extract features and classify it accurately.

Wav2Vec 2.0: The detection framework starts with Wav2Vec 2.0, a deep self-supervised model pre-trained on large amounts of unlabeled speech. Wav2Vec 2.0 turns the raw waveform into contextual speech representations, learning useful acoustic features without needing manually labeled data. This structure helps the model keep important information and capture fine-grained phonetic and prosodic patterns, such as broken phonation, repetitions, and elongated sounds, which are clear markers of dysfluent speech. Because it is pre-trained on large speech corpora, the model already understands general speech features, so adapting it to stutter detection with limited labeled data is easier. Because it extracts contextual features at several levels, it gives a strong base for detecting stutter events with high precision.

Detection and Localization Pipelines: For higher accuracy, the proposed system uses a hybrid framework that combines classification and localization. After preprocessing, the audio runs through three parallel pipelines: classification, transcription, and localization. The classification pipeline extracts speech representations with Wav2Vec 2.0 and passes them through five independent binary classifiers, each trained for a distinct dysfluency type. The localization pipeline finds dysfluency regions at the frame level with Wav2Vec2 frame-level feature extraction, so it can mark the exact segments of speech where dysfluencies occur. Combining the two approaches lets the system catch fine acoustic detail and broader prosodic structure, so results stay stable overall when the recordings vary in noise, clarity, or speaking style.

Classification: Once feature extraction ends, the Wav2Vec 2.0 representations go to the classification stage. Here, five independent binary classifiers give a presence probability for each dysfluency type, using decision boundaries learned during training to separate fluent from dysfluent speech. Sigmoid outputs give per-class probabilities, and binary cross-entropy loss guides optimization during training. Their outputs are combined into

a multi-label result that reports the presence probability of each dysfluency type and summarizes the detected classes and the primary dysfluency. A multitask shared-backbone variant was also trained for comparison, and its outputs are combined with localization at prediction time: the combiner labels each localized region with per-class saliency scores and fuses the two views into a single annotated result. The output gives speech-language pathologists clear results they can act on when identifying stutter events.

Transcription and Alignment: The transcription pipeline uses the Whisper Automatic Speech Recognition (ASR) model to produce a timestamped transcript, with support for English, Kannada, and Hindi. A Connectionist Temporal Classification (CTC) based time alignment step links the pipelines together, mapping the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters keep the alignment accurate for English, Kannada, and Hindi.

Output: In the final stage, the system presents its predictions through a simple interface. The output typically shows the detected dysfluency types along with confidence scores that indicate how certain the model is about each decision, plus a summary of the primary dysfluency. To make the results easier to read, the outputs appear as waveform overlays, spectrogram visualizations, timestamped transcripts, and a detailed clinical-style report. The outputs are designed to be plain, clear, and clinically relevant, so they suit practical use in clinical or self-assessment settings. By providing accurate and interpretable results, the system supports early assessment and can play a useful role in improving outcomes for people who stutter.

Both the web and desktop applications load trained models through the centralized model registry. The registry reads from a registry module (the model/registry package) and a configuration file (registry.json), so checkpoints can be updated or replaced without changing application code, which keeps the system flexible and easy to maintain.

3.3 FLOWCHART

The process starts with the input audio, recorded or uploaded by the user. The audio is converted to 16 kHz mono WAV with FFmpeg, then padded or truncated to 48,000 samples, three seconds at 16 kHz, so all inputs have the same length.

After normalization, the audio runs through classification, transcription, and localization in parallel.

A CTC-based alignment step combines these outputs and maps the detected dysfluencies to specific words or syllables. The final data becomes clear per-word annotations that appear in the waveform and spectrogram displays and in the analysis report. The stages stay

separate so each can be tested on its own, and every branch receives the same normalized audio.

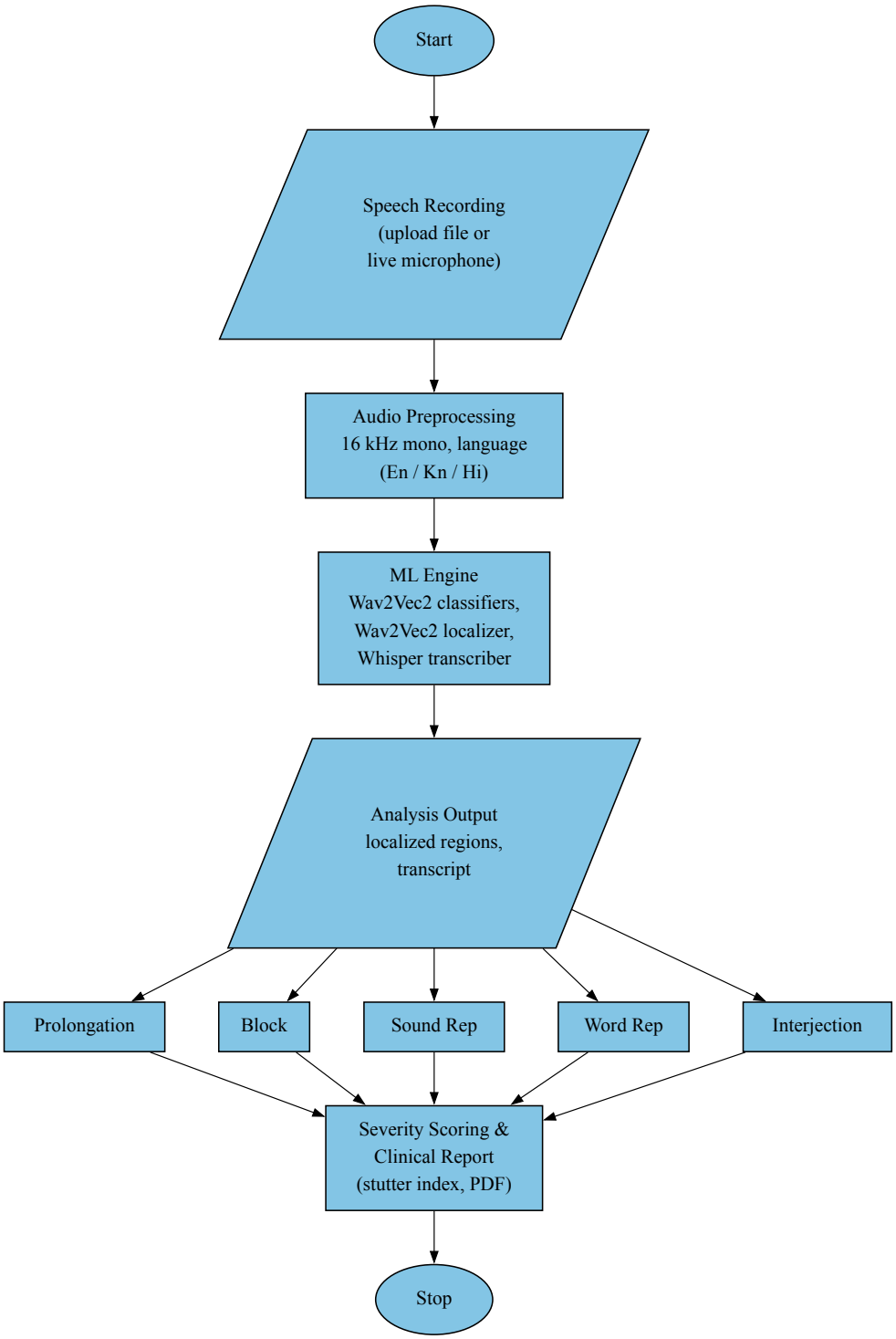


Figure 3.2: Flowchart

Figure 3.2 shows the full workflow, starting from input audio and passing through preprocessing, the three model branches, and the final report. First, the Wav2Vec 2.0 model is used to extract high-level contextual speech features. For better overall performance, a hybrid model combines five separate independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer: the classifiers capture dysfluency-specific acoustic

patterns and the localizer finds fine-grained temporal speech segments. The outputs from both pipelines are fused through CTC-based alignment and passed through a final aggregation layer that sorts the audio samples into blocks, prolongations, sound repetitions, word repetitions, or interjections, along with the precise localized time regions.

The system also defines a structured data flow for training. Training uses three datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset goes through its own normalization functions.

The processed datasets merge into a unified format: a combined_labels.csv file with multi-label binary annotations, plus individual interval CSV files for each audio clip. The merged dataset is split into training, validation, and testing subsets with an 80:10:10 split, with the training subset further re-split 80/20 during model training. The subsets are organized with symbolic links for efficient access, and preprocessed audio files are cached to speed up training.

3.4 USE CASE DIAGRAM AND DESCRIPTIONS

Figure 3.3 shows the use case diagram in UML: the actor stands outside the system boundary, and lines connect it to the use cases it can start. The only actor is the User, since clinicians and people assessing their own speech follow the same path.

Upload / Record Speech is the entry point. Audio can come from the microphone or a saved file, and the interface checks the file before sending it to the backend.

Select Language sets the language used for transcription and alignment; English, Kannada, and Hindi are supported.

Analyze Speech is the central use case. Full mode runs preprocessing, classification, transcription, localization, and CTC-based alignment. Classification-only mode runs the five classifiers and skips transcription and localization. Full mode returns a confidence score for each dysfluency type, a summary of the primary dysfluency, and the time regions where events were found.

View Analysis Results shows the per-class scores, a waveform with color-coded dysfluency regions, a spectrogram, and a timestamped transcript.

Download Clinical Report writes the stored result to a PDF without rerunning the models. Review Assessment History reopens earlier sessions from IndexedDB and localStorage, so recordings can be compared over time. Theme switching and the standardized reading passages support the same workflow.

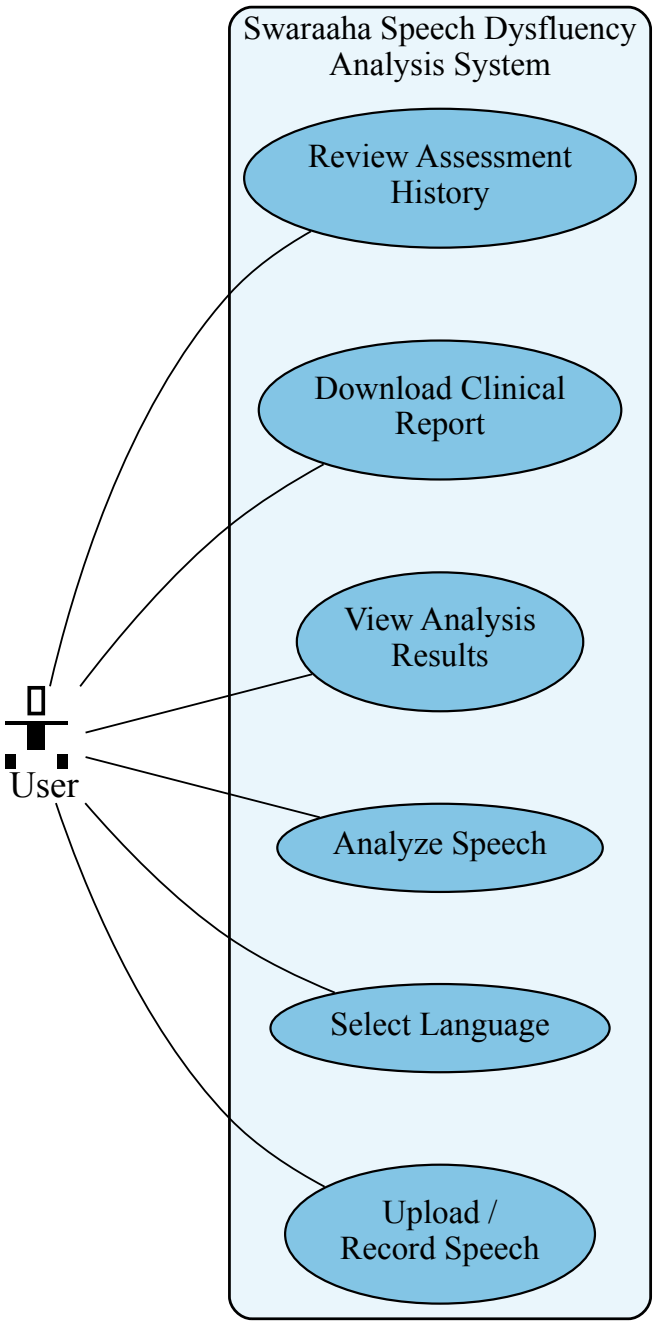


Figure 3.3: Use Case Diagram

Figure 3.3 groups the six use cases around the single actor. Upload / Record Speech and Select Language supply the audio and the language setting. Analyze Speech uses both and produces the classification scores, localized regions, and transcript. View Analysis Results, Download Clinical Report, and Review Assessment History then read the stored result. A report or a history entry cannot exist before an analysis has run. Because the results page, the report, and the history all read the same payload, they show the same outcome for a given recording.

3.5 WORKFLOW DIAGRAM

A workflow diagram is a visual representation of a sequence of operations or tasks that complete a specific process, and people often use one to analyse, design, or manage complex systems. It uses standard symbols: rectangles to mark actions, diamonds to mark decision points, and arrows to show the direction of flow. These make the procedural steps easier for a reader to follow. In the field of deep learning and stutter detection, the workflow diagram shown here maps out the full process for developing a stutter classification and localization system.

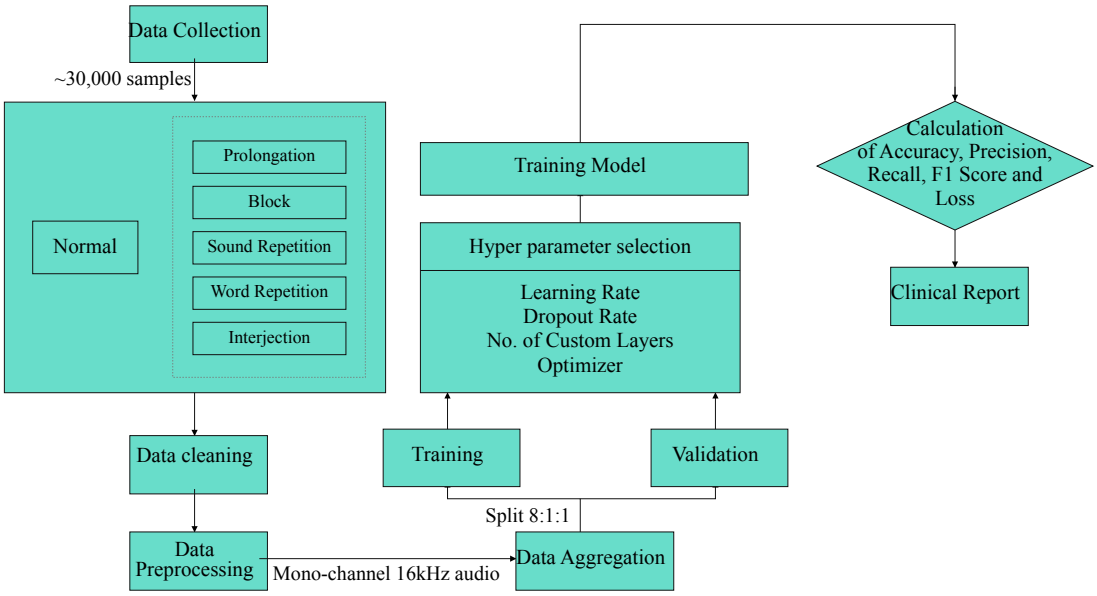


Figure 3.4: Workflow Diagram

Figure 3.4 shows the full workflow of the proposed deep learning pipeline for automatic stutter detection and localization. The process starts by collecting raw speech audio samples from the three public stuttering datasets, Project Boli, SEP-28K, and UCLASS, all converted to 16 kHz mono and standardized through DC offset removal, peak normalization, silence trimming, and fixed-length padding. Data augmentation such as random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling is applied to the waveforms, along with time and frequency masking on the spectrograms, which expands the effective dataset so the models generalize well across varied speaking patterns. The processed audio is split into training, validation, and testing sets at an 80:10:10 ratio, and the training subset is further re-split 80/20 during training. Model development starts by fine-tuning a pre-trained Wav2Vec 2.0 backbone, then builds a hybrid architecture that combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer to capture both global dysfluency patterns and fine-grained temporal segments. Hyperparameters including learning rate, backbone freeze

duration, optimizer, and the number of trainable layers are tuned to improve performance, and the trained network is validated periodically to keep learning stable and prevent overfitting. Finally, evaluation metrics such as precision, recall, F1-score, AUROC, and mean IoU are computed, and the best-performing model is selected for reliable automated assessment of speech dysfluencies.

3.6 SEQUENCE DIAGRAM

The sequence diagram shows how the components interact during execution. The user works through the graphical user interface (GUI), which sends a request to the backend API endpoint (/api/analyze).

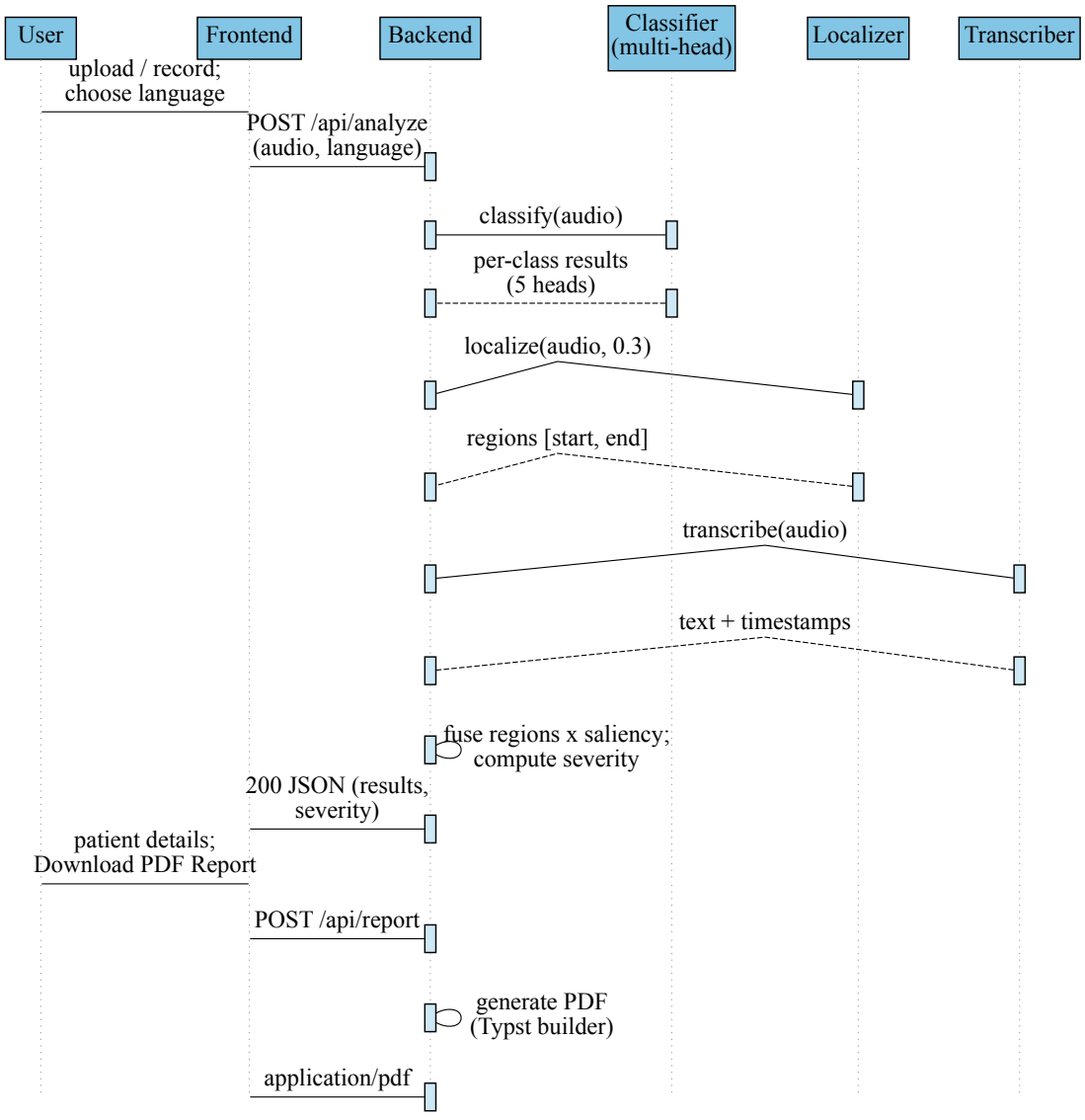


Figure 3.5: Sequence Diagram

The backend sends the request through a series of services: preprocessing, classification, transcription, localization, and alignment. Each service completes its step and passes the result to the next stage. When processing finishes, the results return to the frontend, where the user can see them. The system also saves the results for report generation and history

management. This service sequence gives each component a clear role and makes failures easier to trace. It also lets the system reuse the stored analysis when the user opens a previous result, because the report layer receives the same structured payload that was produced during the original request.

The request lifecycle starts when the user confirms an analysis, at which point the frontend packs the audio, the selected mode, and any optional parameters into a single request. The backend checks the payload before it calls any model, rejecting unsupported formats or empty inputs early so that expensive inference is never wasted on bad data. Preprocessing normalizes the waveform and produces the log-mel spectrograms and frame-level features that later stages expect, and it is the one place where audio conventions are enforced. Classification then gives probability scores for each dysfluency type, and transcription produces a timestamped transcript that the alignment stage later links to the detected events. Localization runs in parallel with transcription where possible, since the two stages share the same preprocessed audio and do not depend on each other's output, which keeps the overall latency low. The alignment stage joins word timings with the localized regions so that each dysfluency can be reported in the context of the spoken words around it. If any single service fails, the backend returns a structured error instead of a partial result, and the frontend shows the failure without discarding the user's audio, so the request can be retried. When the request succeeds, the fused payload is stored locally so that reports and session history can be rebuilt later without re-running models, which also keeps repeated report generation fast and independent of the analysis services. This ordering also keeps the application responsive to users. Independent operations can run in parallel after preprocessing, while the final report waits only for the outputs needed for alignment and presentation. The stored result keeps the predictions, timestamps, transcript, and derived annotations together, so the web and desktop interfaces can show the same analysis without applying different interpretation rules.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

System implementation is a broad and demanding task: it builds and maintains a reliable software architecture that can handle many user interactions, data processing tasks, and model inference operations. Careful design and coding practices keep the application scalable, reliable, and secure, so it can still handle changing workloads and protect sensitive patient data at every stage. Iterative testing and debugging are a central part of the implementation process, so any issues or bugs that arise can be found and fixed before release. This repeated cycle gives users a smooth experience, with few disruptions or errors in their daily use. After deployment, regular monitoring and maintenance check system performance and respond to new requirements as they arise. This steady upkeep keeps the system tuned and still responsive to user needs and expectations as conditions change over time, which supports lasting success and user satisfaction.

4.2 ALGORITHM

The inference process follows a fixed sequence of steps that turns raw speech input into a complete, annotated analysis result. Each step has a defined input and output, which limits the amount of hidden state passed between services. This structure is important for repeatable analysis because the same conversion, normalization, model execution, and reporting stages are used for both recorded and uploaded speech.

- Step 1, Input Acquisition: the system acquires speech input either through real-time recording or by uploading an audio file.
- Step 2, Audio Conversion: the input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency across all inputs.
- Step 3, Preprocessing: the audio is cleaned by removing the DC offset, applying peak normalization to a target amplitude, and trimming silent segments.
- Step 4, Length Normalization: the processed audio is adjusted to a fixed length of 48,000 samples (three seconds at 16 kHz) by padding or truncating as required.

- Step 5, Parallel Processing: the system processes the audio simultaneously through three independent branches:
  1. Classification: Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers, one per dysfluency type, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
  2. Transcription: the Whisper model generates a timestamped transcript of the speech in English, Kannada, or Hindi.
  3. Localization: the Wav2Vec2 frame-level localizer identifies the frame-level regions within the audio where dysfluencies occur.
- Step 6, Alignment: the detected dysfluency regions are mapped to the corresponding words or syllables in the transcript using CTC-based time alignment.
- Step 7, Combination and Severity: the combiner labels each localized region with per-class saliency scores from the classifier and fuses the classification and localization views, and the severity module computes a stutter index from the ratio of dysfluent speech duration to total speech duration.
- Step 8, Visualization and Report Generation: the system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, and generates a detailed clinical-style report.

4.3 IMPLEMENTATION REQUIREMENTS

This section lists the implementation requirements for detecting and localizing stuttering with a hybrid deep learning model that combines Wav2Vec 2.0 feature extraction, five independent Wav2Vec 2.0 binary classifiers, a Wav2Vec2 frame-level localizer, and Whisper automatic speech recognition. The main goal is to build a reliable, efficient system that finds speech dysfluencies and localizes them within the audio. The hybrid model uses the strengths of both architectures: Wav2Vec 2.0 supplies self-supervised, contextual speech representations that capture fine-grained phonetic and prosodic patterns, while Whisper provides sequence-level transcription that helps map localized dysfluency regions to specific words and syllables.

The implementation relies on the Python ecosystem, which provides an interactive environment for model development, experimentation, and analysis. Python is the main language used for backend development because it is versatile and also supports a very wide range of scientific and deep learning libraries such as PyTorch, the Hugging Face Transformers library, and NumPy, which help with model construction, training, and

evaluation. For the frontend, React 19 and PySide6 build a simple web interface and desktop application where users can record or upload speech audio and view classification and localization results. A central model registry manages all trained models. It is configured through a registry.json file that maps each task to its checkpoint and per-class thresholds, so checkpoints can be updated or replaced without changing application code. Python also handles matrix manipulation, data visualization, algorithm implementation, and smooth integration with other programming environments, which makes it a good platform for developing and testing the hybrid model for stuttering classification and localization.

4.3.1 Train the Model

Training is a major step in developing the system for stutter classification and localization. The speech audio dataset is first collected from Project Boli, SEP-28K, and UCLASS, then preprocessed through conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding. Data augmentation also improves audio quality and reduces overfitting risk. The Wav2Vec 2.0 model is then trained to extract high-level contextual speech features, using its self-supervised pretraining to learn strong representations. To raise classification accuracy further, the hybrid model combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer. The localizer captures fine-grained temporal features at the frame level, adding detail beyond segment-level extraction of the classifiers. CTC-based time alignment then combines the outputs of both models into one feature representation. Training uses the AdamW optimizer to adjust learning rates adaptively and keep convergence stable. It also uses mini-batches of audio, and backpropagation updates the weights from the computed loss. Validation runs periodically to monitor accuracy and prevent overfitting. After training, the system saves optimized parameters and weights, so the models can predict on new speech audio without retraining.

4.3.2 Test the Model

Testing comes after training to check how well the proposed model detects stutter events in unseen speech audio. When a user records or uploads a sample through the web interface or desktop application, it goes through the very same preprocessing as training: conversion to 16 kHz mono WAV, DC offset removal, peak normalization, silence trimming, and fixed-length padding. This keeps input uniform for accurate analysis. The trained Wav2Vec 2.0 model then extracts high-level contextual speech features. For higher accuracy, the audio also passes through the hybrid model, which combines five independent Wav2Vec 2.0 binary classifiers with a Wav2Vec2 frame-level localizer

and captures both segment-level dysfluency patterns and fine temporal detail. CTC-based time alignment combines the features from both models into one representation. The final aggregation layer then produces a classification result that names the dysfluency types present (blocks, prolongations, sound repetitions, word repetitions, or interjections) and the exact regions involved. The interface shows the outcome with waveform overlays, spectrograms, a timestamped transcript, and confidence scores. The system also compares the model predictions with ground truth labels to measure accuracy and generalization. This phase shows that the hybrid model performs well on new and unseen samples, which points to reliable stutter classification and localization in real clinical use.

4.3.3 Model Evaluation

Model evaluation is essential for assessing the overall performance and reliability of the proposed system. Once the models are trained and tested, several quantitative metrics measure their classification performance:

- Accuracy: measures the proportion of correctly predicted samples.
- Precision: the share of predicted positive instances that are actually positive.
- Recall (Sensitivity): measures how effectively the model identifies all relevant positive samples.
- F1-Score: provides a harmonic mean of precision and recall to balance both metrics.

The five Wav2Vec 2.0 binary classifiers, one per dysfluency type, are further measured with AUROC (Area Under the Receiver Operating Characteristic Curve), AUPRC (Area Under the Precision-Recall Curve), and specificity, summarized per class and as a macro average across the five dysfluency classes. The Wav2Vec2 frame-level localizer is measured with frame-level precision, recall, and F1, detection accuracy, mean Intersection over Union (IoU) between predicted and ground-truth regions, and the false alarm rate per minute.

A confusion matrix for each of the five binary classifiers shows the distribution of predictions, including true positives, false positives, true negatives, and false negatives. This evaluation shows where each class is strong or weak; interjections, for example, are detected most reliably, while blocks are the hardest.

All reported results use the held-out test set of 3,715 clips, which is never touched during training or threshold selection, together with a cross-corpus Boli set that checks how well the models transfer to unseen speakers.

4.4 USER INTERFACE

The web-based interface connects the user to the proposed stutter detection and localization system. Users record speech or upload audio files for analysis. Once audio is provided, the backend system carries out the preprocessing steps, followed by feature extraction, classification, transcription, and localization using the trained models. Initially, the Wav2Vec 2.0 model is used to extract high-level contextual speech features, and to achieve higher accuracy, the hybrid pipeline combining five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer is employed. Within a few seconds, the system presents the predicted result on the screen for each of the five dysfluency types, along with confidence scores, the localized regions, and the timestamped transcript. The tool is designed for clinicians, speech-language pathologists, researchers, and individuals who stutter.

The system provides two interfaces to the same analysis pipeline: a web application and a desktop application. The web frontend is built with React 19, TypeScript, and Tailwind CSS, with a sidebar layout covering the input page, results page, history, and reading passages. The input page supports audio file upload (WAV, MP3, FLAC, M4A, OGG, WMA) and direct microphone recording through the MediaRecorder API, with client-side validation before the file is sent to the backend as multipart form data. The results page shows classification results with confidence scores, an interactive waveform with color-coded dysfluency region overlays, a timestamped transcript, and a button to generate and download a PDF report. The backend runs on FastAPI served through Uvicorn, exposing the /api/classify, /api/localize, /api/analyze, and /api/report endpoints. Assessment history stays in the browser: audio files go into IndexedDB for offline access and analysis metadata into localStorage.

The desktop application is built with PySide6 (Qt for Python) for a native cross-platform interface. It includes modules for audio recording and playback (sounddevice), file handling (soundfile), and model inference, calling the shared model package directly with no separate server, so it works fully offline. PDF report generation uses Typst with pypdfium2 for viewing.

Both interfaces share the same model package, preprocessing conventions, and result schema, so a recording analysed in the browser and the same recording analysed on the desktop produce identical predictions and reports. The web client keeps long requests responsive by streaming progress updates and blocking any duplicate submissions while an analysis runs, while the desktop client runs inference in a worker thread so the window still stays interactive during transcription and localization. Theme switching, keyboard

navigation, and consistent layout across window sizes are handled at the component level, which keeps the interface usable for clinicians who move between machines and screen sizes. The client checks uploaded audio before it ever reaches the backend, and the results view keeps the waveform, transcript, and per-class confidence table in sync so that selecting a timestamp jumps to the exact matching part of the recording. Report generation is kept separate from analysis: the stored result payload is enough to rebuild the PDF, so a report can still be regenerated with a different layout or header without repeating the expensive model inference. This shared design also makes maintenance easier, because a change to the result schema or the report template reaches both interfaces from one single source instead of being implemented twice.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Testing is a major part of developing the proposed system for stutter detection and localization, since it checks the overall accuracy, stability, and reliability of the individual models and the whole hybrid pipeline. Roughly one percent of people worldwide stutter, and diagnosis today still depends mostly on manual assessment by speech-language pathologists. That work is subjective, time-consuming, and not available to everyone in practice, so early and reliable detection matters. In this research, Wav2Vec 2.0-based models were first used to classify speech dysfluencies from audio. To improve diagnostic precision further, the hybrid framework combines five independent Wav2Vec 2.0 binary classifiers with the Wav2Vec2 frame-level localizer and with Whisper automatic speech recognition, and it handles classification, localization, and transcription in a single pipeline. The testing process covered every stage of the entire pipeline, including data preprocessing, model training, classification, localization, and web-based visualization, through unit, integration, functional, and system tests. Testing showed that the proposed system meets all functional and non-functional requirements and still classifies and localizes speech reliably. The models were judged on precision, recall, F1-score, AUROC, AUPRC, specificity, and confusion matrix analysis, and also on a held-out test set and a cross-corpus set, which showed their reliability and ability to generalize. The results are broken down by model family and dysfluency class, and the two model families, the Wav2Vec2-based models and the CNN-based models, are compared against each other and analyzed for their strengths and weaknesses.

5.2 TYPES OF TESTS

Four types of tests were performed: unit testing, integration testing, functional testing, and system testing. Together they verify the correctness of individual components, the interfaces between them, the complete feature set, and the behavior of the finished system.

5.2.1 Unit Testing

Unit testing isolates the smallest blocks of code, specific functions and classes, to verify they work as expected. The project uses pytest suites organised under model/tests, backend/tests, app/tests, and shared/tests.

Unit tests verify the preprocessing steps (sample-rate conversion, DC offset removal, peak normalization, silence trimming, and fixed-length padding), dataset normalization and the merged combined_labels.csv format, model registry checkpoint loading, the combiner and CTC alignment logic, and the computation of the evaluation metrics. On the desktop side, individual widgets such as the waveform view, results panel, and audio handler are tested in isolation. Ruff enforces consistent code style across the codebase.

5.2.2 Integration Testing

Integration testing checks that the software modules work together once connected. The backend tests exercise the FastAPI endpoints (/api/analyze, /api/classify, /api/localize, and /api/report), which chain preprocessing, classification, transcription, localization, and fusion into a single analysis response. The classification, fusion, localizer, severity, and report generator services are tested as combined units, and the model registry is checked against the classifier and localization pipelines. The desktop application calls the shared model package directly, and its model runner is integration-tested against the same service layer.

5.2.3 Functional Testing

Functional testing checks the full set of features against the specifications under typical and edge cases. Testers exercise the workflow of recording or uploading audio, running the analysis, and viewing the results page with classification scores, waveform overlays, and the timestamped transcript end to end. Testers verify report generation and download, severity mapping, multilingual transcription for English, Kannada, and Hindi, and history storage through IndexedDB and localStorage against all the requirements defined in Chapter 2.

5.2.4 System Testing

System testing puts the whole software product through end-to-end scenarios in conditions that resemble the user environment. Testers ran the React web application in supported browsers and the PySide6 desktop application in a standalone offline setup with no separate server, both against the same machine learning backend. Testers checked the supported audio formats (WAV, MP3, OGG, and M4A) across the upload, drag-and-drop, and microphone capture paths, confirmed that corrupted files were rejected with clear error messages, and checked that analysis turnaround stayed within a few seconds per clip as performance requirements in Chapter 2 demand. The full system-level suite showed that the application meets its functional and non-functional requirements.

5.3 RESULT AND ANALYSIS

This section presents the evaluation of the trained models: how the Wav2Vec2-based and CNN-based model families were trained and evaluated, the hyper parameter tuning applied, and a comparative performance analysis of all architectures.

5.3.1 Model Evaluation of the Wav2Vec2 Classifiers

The model registry defines two Wav2Vec2-based classification configurations: the default single configuration of five independent Wav2Vec2 binary classifiers, and a multitask variant that shares a single backbone and five classification heads. This section first analyzes the multitask variant in detail and then reports the shipped single configuration; the confusion matrices at the end of the section come from the shipped single configuration.

The multitask variant uses a single Wav2Vec2-base backbone with five shared binary heads. Training used Focal Loss (gamma=2.0), backbone freezing for 3 epochs, AdamW (LR 3×10^{-5}), a 500-step warm-up, and early stopping with a patience of 5. Results on 3,715 held-out test samples at the default threshold of 0.5 are in Table 5.1.

Class	**Precision**	**Recall**	**F1**	**AUROC**	**AUPRC**	**Support**
Prolongation	0.566	0.440	0.495	0.845	0.491	400
Block	0.584	0.264	0.364	0.768	0.444	553
Sound Repetition	0.705	0.502	0.586	0.878	0.624	494
Word Repetition	0.621	0.399	0.486	0.855	0.550	366
Interjection	0.810	0.746	0.776	0.935	0.848	834
Macro Average	–	–	**0.542**	–	–	**3,715**

Table 5.1: Multitask Wav2Vec2 classifier results

The per-class support counts cover positive samples; the macro row reports the total of 3,715 test clips because a clip may contain several stutter types at once.

The macro F1 at the default threshold is 0.542. Interjection performs best (F1=0.776, AUROC=0.935), which makes sense because filler words like “um” and “uh” have distinct acoustic patterns. Block is the hardest class (F1=0.364); silent pauses and hesitations are hard to distinguish from normal speech pauses.

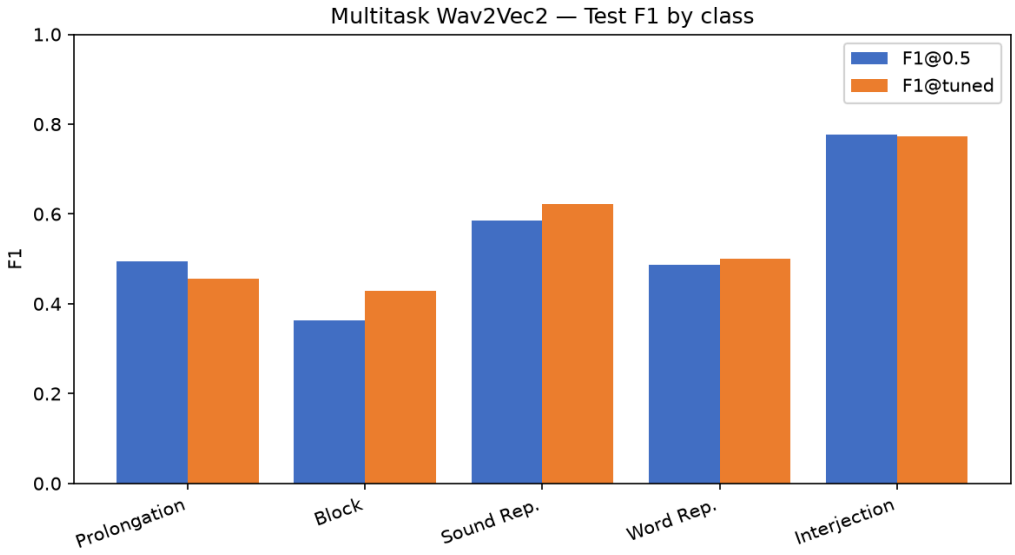


Figure 5.1: Per-class F1 of the multitask Wav2Vec2 classifier

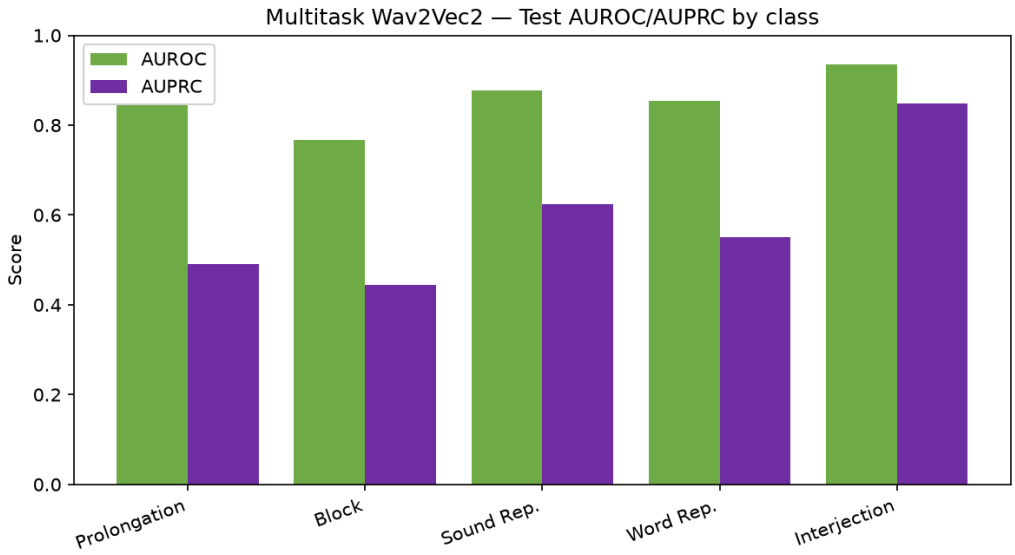


Figure 5.2: Per-class AUROC and AUPRC of the multitask Wav2Vec2 classifier

The per-class F1 scores are plotted in Figure 5.1, and the corresponding AUROC and AUPRC values are plotted in Figure 5.2. Both figures mirror the table above: interjection leads on every measure, while block trails on F1 even though its AUROC and AUPRC stay reasonably high.

The shipped single configuration of five independent binary classifiers scores slightly higher in distribution, with a test F1 of 0.573 at threshold 0.5 (0.570 with tuned thresholds).

The confusion matrices for the five dysfluency types are shown in Figure 5.3 and visualize the distribution of predictions of the shipped single configuration of five independent binary classifiers.

Before reading the matrices, two details about how the aggregate scores were obtained matter here. Each binary classifier was trained with a class-balanced objective, and the operating threshold was selected on the validation split rather than the test split, so the matrices show the behaviour of the shipped configuration at its default threshold, not a threshold deliberately tuned to inflate the reported numbers. Reading them row by row reveals how often each event type is correctly recovered, while reading them column by column shows which fluent or other-type segments are most likely to be mistaken for it. This distinction matters in practice, because a clinician cares both about missed events and about false alarms that interrupt a session.

The matrices are arranged so that the true label is on one axis and the predicted label on the other, with darker cells indicating larger counts. A classifier that simply predicted the majority class every time would produce a single dark row. None of the five matrices does, so each head learned a real decision boundary instead of collapsing to the dominant label. The remaining off-diagonal mass is concentrated in specific cells, which points to concrete confusions that later sections address through threshold tuning and post-processing.

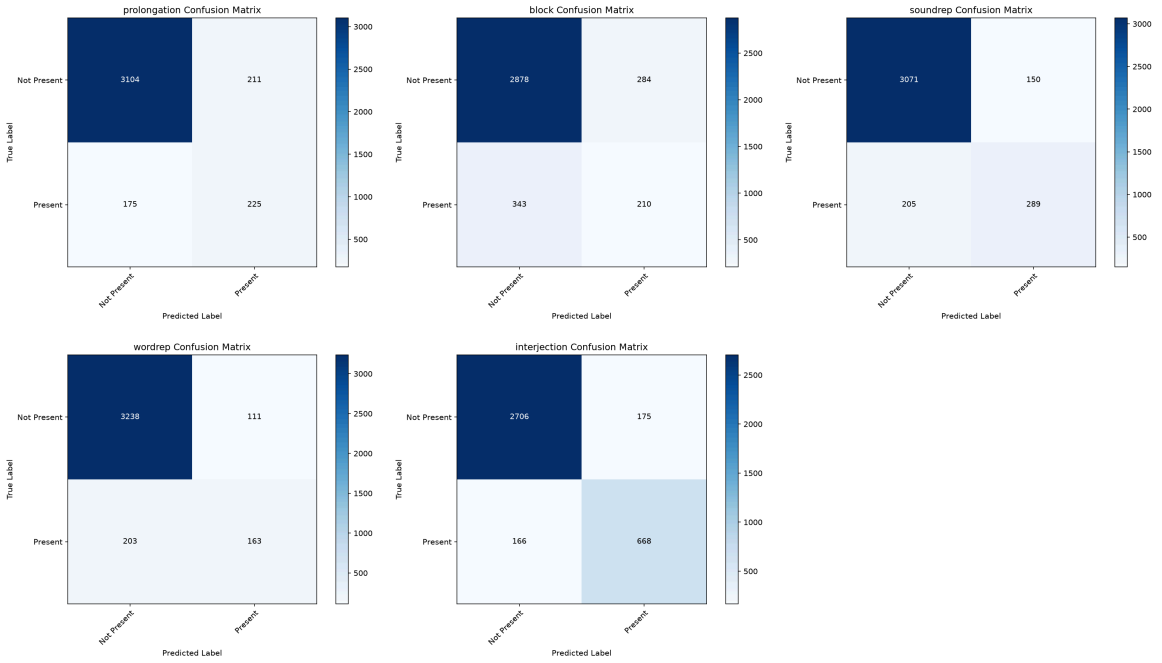


Figure 5.3: Confusion matrices for the five binary classifiers

Each confusion matrix contrasts the true labels against the predicted labels for one binary classifier, so the diagonal cells count correct decisions and the off-diagonal cells expose the direction of error. A strong diagonal with a light upper-right corner means the classifier rarely over-triggers, while a heavy lower-left corner indicates missed events, and the balance between the two is what separates a conservative detector from an over-sensitive one. Interjection shows the cleanest diagonal, consistent with its high F1 and AUROC,

because its acoustic signature, typically a short vocalized filler, is spectrally distinct from fluent speech. Block and word repetition show more spread, since prolonged silences and repeated words can resemble pauses and normal hesitation, which the classifiers then confuse at the boundary. Reading the matrices alongside the per-class table makes the trade-off explicit: a classifier tuned for high recall shifts mass into the false-positive corner, while one tuned for precision leaves more events undetected. This is precisely why the shipped system keeps separate binary classifiers instead of a single multi-class head, since each event type can be calibrated to its own operating point without disturbing the others.

The Wav2Vec2-based model family also provides the frame-level localizer, evaluated on the same 3,715 test samples at a threshold of 0.5; the shipped registry inference threshold for this localizer is 0.3. Results are in Table 5.2.

Metric	**Value**	**Metric**	**Value**
Frame Precision	0.805	Detection Accuracy	0.586
Frame Recall	0.738	Mean IoU	0.779
Frame F1	0.770	False Alarms/min	4.28
Frame Specificity	0.847	Predicted Events	1,846

Table 5.2: Wav2Vec2 localizer evaluation results

The localizer detects most dysfluency events with good accuracy: frame precision is 0.805 and recall is 0.738, so most true events are found and most predictions are correct. The mean IoU of 0.779 means the regions it finds overlap well with ground truth. Of 995 true events it predicted 1,846, of which 1,263 were false alarms (4.28 per minute).

The gap between frame-level and event-level behaviour matters in practice. A frame F1 of 0.770 means that most individual frames are labelled correctly, but the event count shows that the localizer predicts nearly twice as many events as are actually present, so a clinician using the raw output would see clusters of short, spurious detections around genuine events. The 4.28 false alarms per minute figure is the practical consequence of that over-prediction, and it is the main reason the shipped registry applies a higher inference threshold of 0.3 rather than the 0.5 used for reporting. Raising the threshold trades recall for precision by removing low-confidence detections, which reduces the false-alarm rate at the cost of occasionally dropping a weak but real event. The choice of threshold is therefore a clinical decision as much as a technical one, and the system exposes it as a configurable parameter so that a screening workflow can favour sensitivity while a detailed assessment can favour precision. The following subsection turns to how

the classifiers behind these detections were trained, since the localizer and the classifiers share the same pretrained backbone and the same preprocessing conventions.

Figure 5.4 plots the localizer's key frame-level metrics.

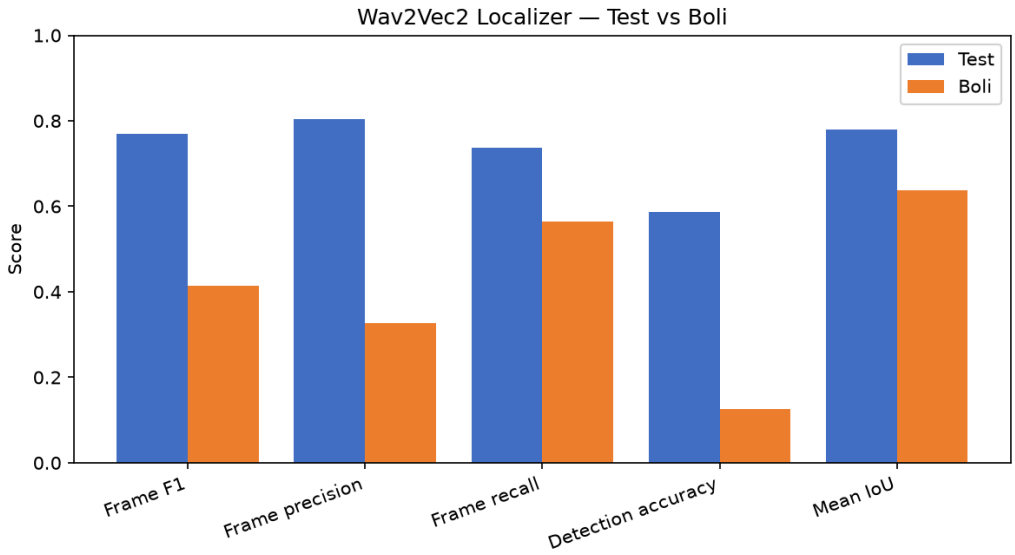


Figure 5.4: Frame-level metrics of the Wav2Vec2 localizer

5.3.2 Wav2Vec2 Classifier Training

The training phase prepares the models that the system relies on for classification and localization.

The system is trained and evaluated on three publicly available stuttering datasets: Project Boli, SEP-28K, and UCLASS. Project Boli is a multilingual Indian dataset with stuttering data in Hindi, Kannada, Telugu, and other Indian languages. It comes from GitHub and includes both read and spontaneous speech. The ingestion pipeline keeps only the English task codes from this dataset, so the Boli material used here is English speech. It is therefore a cross-corpus set, not a cross-lingual one. SEP-28K offers roughly 28,000 audio clips from stuttering support group podcasts, annotated for blocks, prolongations, sound repetitions, word repetitions, and interjections, and it comes from Kaggle. UCLASS is another stuttering speech dataset from Kaggle, and it adds supplementary annotated clips. All three datasets are normalized into one format with a combined_labels.csv file of multi-label binary annotations and interval files per clip. The merged dataset is split 80:10:10 into training, validation, and testing subsets, and during training the training subset is re-split 80/20 by a stratified split into a fine-tuning set and a validation set used for early stopping and threshold selection.

Each classifier is trained in a two-stage scheme. The backbone stays frozen for the first three epochs so the classification head can stabilize, then it is unfrozen with a learning

rate scaled to 0.1 of the head learning rate. Focal Loss with a gamma value of 2 handles the class imbalance, and the AdamW optimizer runs at a learning rate of 3×10^{-5} with a weight decay of 0.01 and a 500-step warm-up. Early stopping with a patience of 5 epochs prevents overfitting, and the checkpoint with the highest macro F1-score on the validation set is saved as the final model. During training, data augmentation applies waveform-level transforms (random noise injection, time stretching, pitch shifting, temporal shifting, and amplitude scaling) and spectrogram-level transforms (time and frequency masking). Checkpoints are stored under names that encode their training hyperparameters through a fingerprint convention, with checkpoint-based resume, an audio preprocessing cache for fast re-runs, torch.compile for roughly twice the training speed on CUDA, and gradient accumulation for effective batch size scaling.

We found and fixed several training problems. BCEWithLogitsLoss with a single logit per class created a zero-gradient equilibrium: the model simply collapsed to always predicting the majority class. Switching to CrossEntropyLoss with two logits (present and not present) fixed this. Gradient clipping in the classifier loop silently stopped learning by scaling the classifier head gradients down, so it was removed from the classification loop while the localizer loops kept it. Mixed-precision training with a GradScaler produced NaN gradients; using torch.autocast alone worked. Freezing the backbone for 3 epochs let the classifier head stabilize before fine-tuning, and longer freeze durations hurt performance.

5.3.3 Model Evaluation of the CNN-based Models

The CNN-based models form the second model family, built to run classifications with far fewer parameters and to compare generalization against the pre-trained Wav2Vec2 baseline.

Four CNN-based multitask classification architectures were evaluated: CNN Pool, CNN Single, CNN-LSTM, and CNN-Transformer, ranging from 0.27M to 0.54M parameters. On the held-out test set they score markedly lower than the Wav2Vec2 classifiers, with tuned F1 values between 0.249 and 0.271. On the cross-corpus Boli set, however, the ranking flips: CNN Single reaches 0.476 and CNN-LSTM 0.508, the best cross-corpus result of all seven architectures, suggesting that the lighter models generalize better to unseen speakers.

The CNN spectrogram localizer was evaluated on the same 3,715 test samples at a threshold of 0.5. It is far more permissive than the Wav2Vec2 localizer: frame recall is high (0.873) but precision is much lower (0.527), and it predicted 4,483 events against 995 true events, with 4,329 false alarms (23.49 per minute). Its detection accuracy (0.155) and

specificity (0.330) are correspondingly low. The heavier Wav2Vec2 localizer is therefore shipped as the default; the registry inference threshold is 0.3 for both localizers.

5.3.4 CNN-based Model Training

The CNN-based models are trained on the same unified dataset and with the same two-stage scheme used for the Wav2Vec2 classifiers, but they consume mel-spectrograms instead of raw audio embeddings. CNN Pool applies global pooling over the convolutional feature maps, CNN Single processes a single spectrogram path, CNN-LSTM adds a recurrent layer to capture temporal context, and CNN-Transformer uses a transformer encoder over the spectrogram features. Training also applies standard spectrogram-level augmentation, namely time and frequency masking, alongside the waveform-level transforms. These networks have only 0.27M to 0.54M parameters compared with roughly 94M to 97M for the Wav2Vec2 family, so they train quickly. We used them to study the trade-off between parameter count and cross-corpus generalization.

5.3.5 Hyper Parameter Tuning

A threshold sweep on the validation set tested thresholds from 0.1 to 0.9 in steps of 0.05 for each class. The threshold that maximized F1 for each class was stored in the model registry under the multitask configuration. The results are in Table 5.3.

Class	**Default (0.5) F1**	**Optimal Threshold**	**Tuned F1**	**Improvement**
Prolongation	0.495	0.55	0.457	−7.8%
Block	0.364	0.35	0.429	+17.8%
Sound Repetition	0.586	0.40	0.621	+6.0%
Word Repetition	0.486	0.35	0.499	+2.8%
Interjection	0.776	0.45	0.773	−0.5%
Macro Average	**0.542**	–	**0.556**	**+2.6%**

Table 5.3: Threshold optimization results

Threshold tuning raised the macro F1 from 0.542 to 0.556. Block improved the most (0.364 to 0.429) because the default threshold of 0.5 is too high for a class with strong imbalance. For prolongation and interjection the tuned threshold lowers F1 slightly on the test set, but per-class tuning on the validation set still yields a net gain of +2.6% overall.

Additional hyper parameters were tuned during development. The backbone freeze duration was compared at 3 and 20 epochs; freezing for 3 epochs (MT frz3) clearly beats 20 epochs (MT frz20) on the test set, showing that early unfreezing matters for fine-tuning. The learning rate scaling factor of 0.1 between the head and the unfrozen backbone, a

500-step warm-up, a weight decay of 0.01, and gradient accumulation for effective batch size scaling together gave the most stable training runs.

5.3.6 Performance Analysis of the Models

Seven architectures were compared in total, ranging from the shipped five independent Wav2Vec2 classifiers to various CNN-based multitask models. Test set and cross-corpus Boli results are in Table 5.4.

Architecture	Type	Parame-ters	Test F1 (0.5)	Test F1 (Tuned)	Boli F1 (Tuned)
5x Wav2Vec2	Classifier	94.6M	0.573	0.570	0.292
MT Wav2Vec2 (frz3)	Multitask	97.3M	0.542	0.556	0.238
MT Wav2Vec2 (frz20)	Multitask	97.3M	0.141	0.378	0.214
CNN Pool	Multitask	0.34M	0.149	0.256	0.332
CNN Single	Multitask	0.27M	0.212	0.262	0.476
CNN-LSTM	Multitask	0.46M	0.242	0.249	0.508
CNN-Transformer	Multitask	0.54M	0.269	0.271	0.345

Table 5.4: Comparative study of seven architecture variants

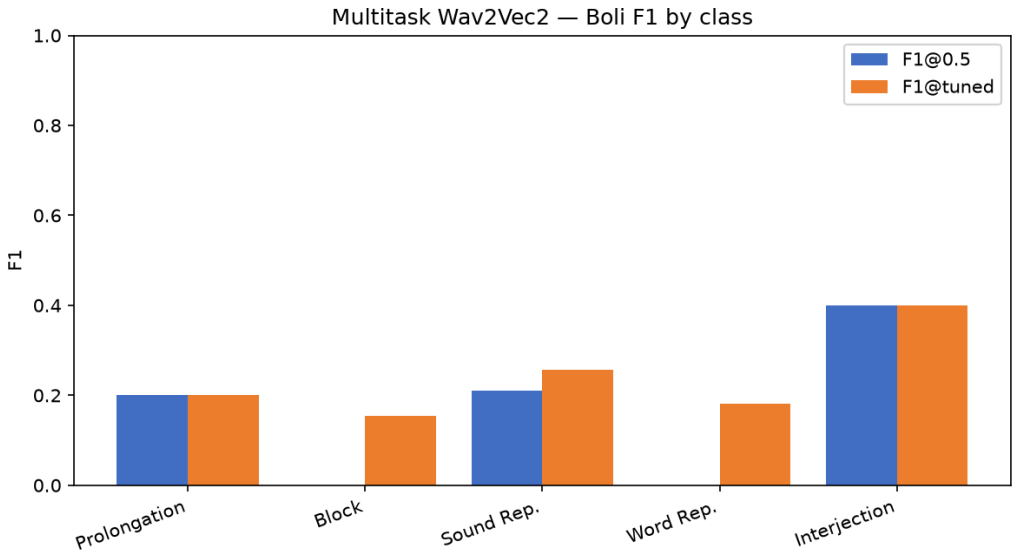


Figure 5.5: Per-class tuned F1 on the Boli set

The shipped five-classifier Wav2Vec2 configuration scores highest in distribution (tuned F1 0.570), narrowly ahead of the multitask variant (0.556); both benefit from large pre-trained speech representations. The CNN models have far fewer parameters but score lower on the test set. On the Boli cross-corpus set the ranking flips: CNN-LSTM reaches 0.508, the best cross-corpus result, followed by CNN Single (0.476). Lighter models

seem to generalize better to unseen speakers, while heavy pre-trained models do better on familiar data.

Figure 5.5 shows the per-class tuned F1 scores on the Boli cross-corpus set, where the lighter CNN variants lead the whole comparison. This result shows why both ordinary test performance and cross-corpus performance need careful reporting. The first measures how well the models recognize patterns similar to those seen during development, while the second shows sensitivity to changes in speaker, language, recording style, and data source.

It was concluded that the Wav2Vec2 family should be shipped for in-distribution accuracy, with the CNN-LSTM variant available as a lighter alternative that transfers best to unseen speakers and languages.

This conclusion reflects a deliberate trade-off rather than a claim that one architecture dominates on every axis. The Wav2Vec2 classifiers benefit from representations learned on large amounts of unlabeled speech, which is why they lead on the test set where the speakers and recording conditions resemble the training distribution. On the Boli corpus, where speakers and languages differ from the podcast-style training data, the advantage narrows and the smaller CNN models transfer better, suggesting that heavy pretrained backbones overfit to the acoustic characteristics of their training corpora. The practical implication is that the system reports both numbers instead of a single headline score, and that deployment should state which regime is expected. For a clinic working with speakers and equipment similar to the training data, the Wav2Vec2 configuration is the better default; for a deployment that must handle unfamiliar speakers or languages without retraining, the CNN-LSTM variant is the safer choice and runs on far more modest hardware.

The comparison also clarifies where further work would pay off: the gap between in-distribution and cross-corpus performance is larger than the gap between architectures, so data coverage and speaker diversity matter more than adding parameters. These findings also guide how the system should be evaluated after deployment. Performance should be monitored separately for familiar and unfamiliar recording conditions, and results should be reviewed by dysfluency class rather than reduced to one overall score. That reporting practice makes changes in data coverage, thresholds, or model architecture visible and helps prevent an improvement for one class from masking a regression in another.

CHAPTER 6

SCREENSHOTS

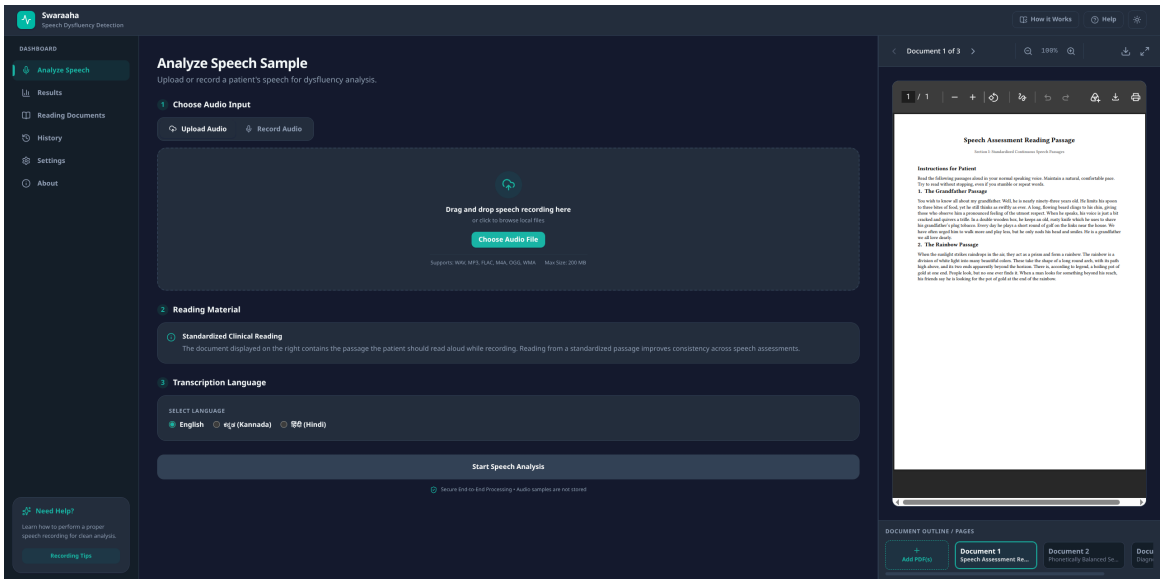


Figure 6.1: Home page of the web interface

The home page of the Swaraaha web application, shown in Figure 6.1, has an Analyze Speech Sample panel where users can upload a supported audio file or record speech directly, choose the transcription language, and then start the analysis. A reading-material viewer also sits alongside the input controls so that users can read a standardized passage before recording.

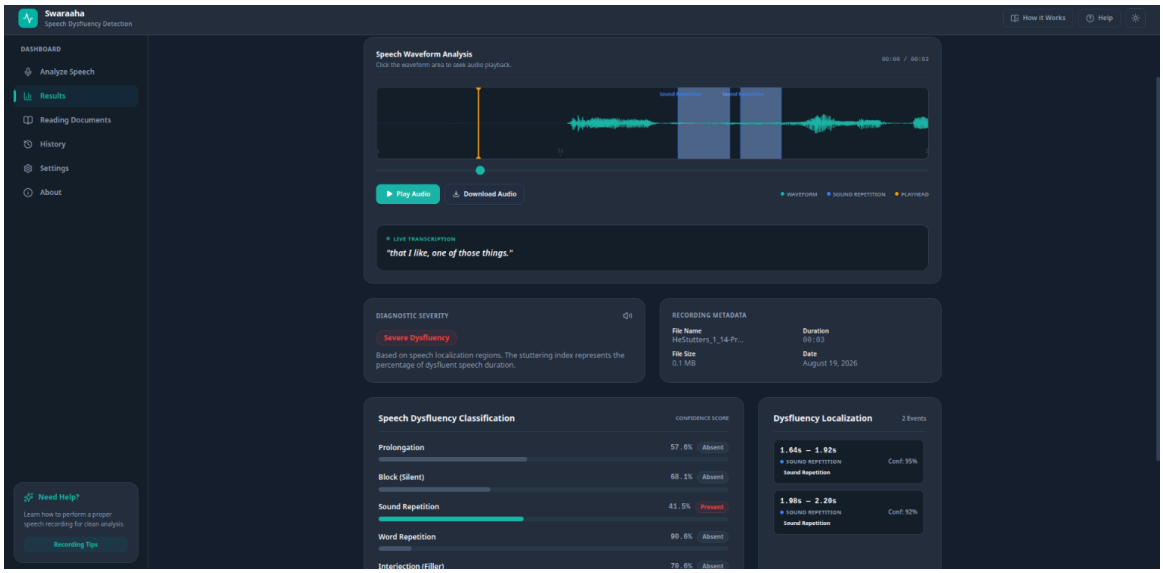


Figure 6.2: Results page of the web interface

The web results page, shown in Figure 6.2, shows the recorded waveform with localized dysfluency regions, synchronized playback controls, a timestamped live transcription, and

confidence scores for prolongation, block, sound repetition, word repetition, and interjection. Users can upload another sample, review the full analysis, or download a PDF report.

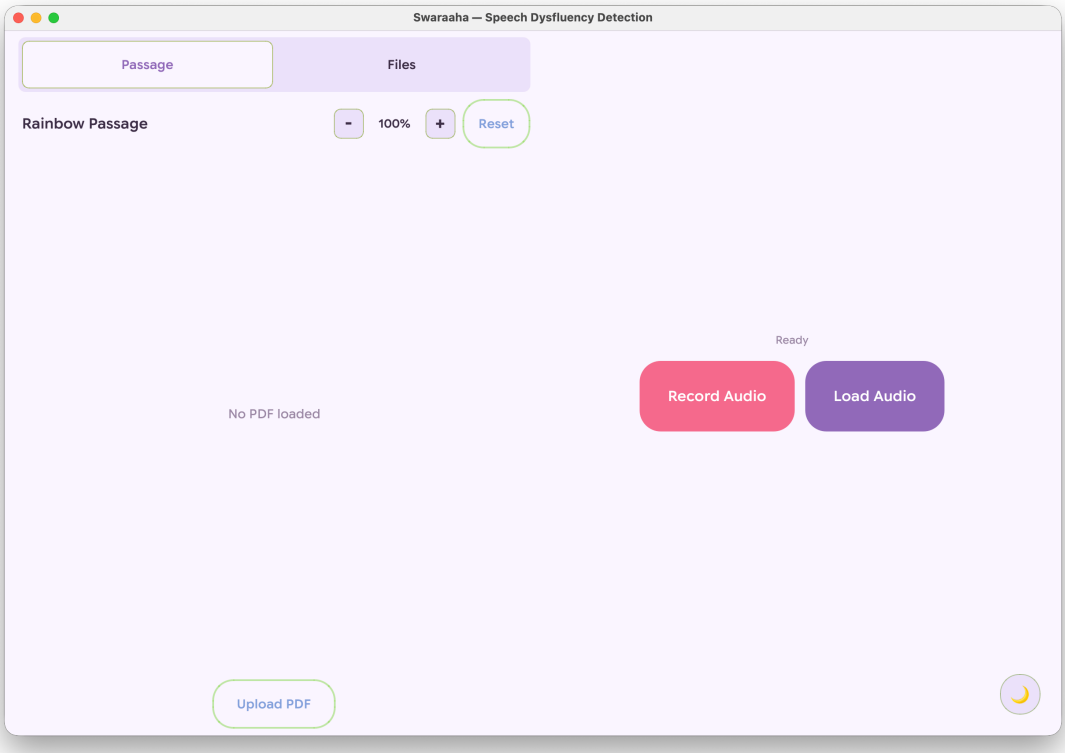


Figure 6.3: Home page of the desktop application

The home page of the desktop application, shown in Figure 6.3, allows users to open a standardized reading passage, load an audio file from the local file system, or record new speech directly. The interface also shows the current processing status and gives the controls needed to continue once an audio sample has been selected. The desktop client closely follows the web workflow but runs entirely on the local machine, so it still works where a network connection is unreliable or where the audio cannot leave the clinic. The reading passage viewer sits right in the same window as the input controls, so a speaker can read the standardized text and start recording without switching applications, which reduces the variability that different reading setups introduce. The status indicator reports each processing stage in turn, and the controls stay disabled until a valid audio sample is available, which prevents accidental submissions and keeps the interaction predictable for non-technical users. Because the model package is imported directly instead of called over HTTP, the desktop application avoids the overhead of serializing audio and results, and it still works when the web backend is not running.

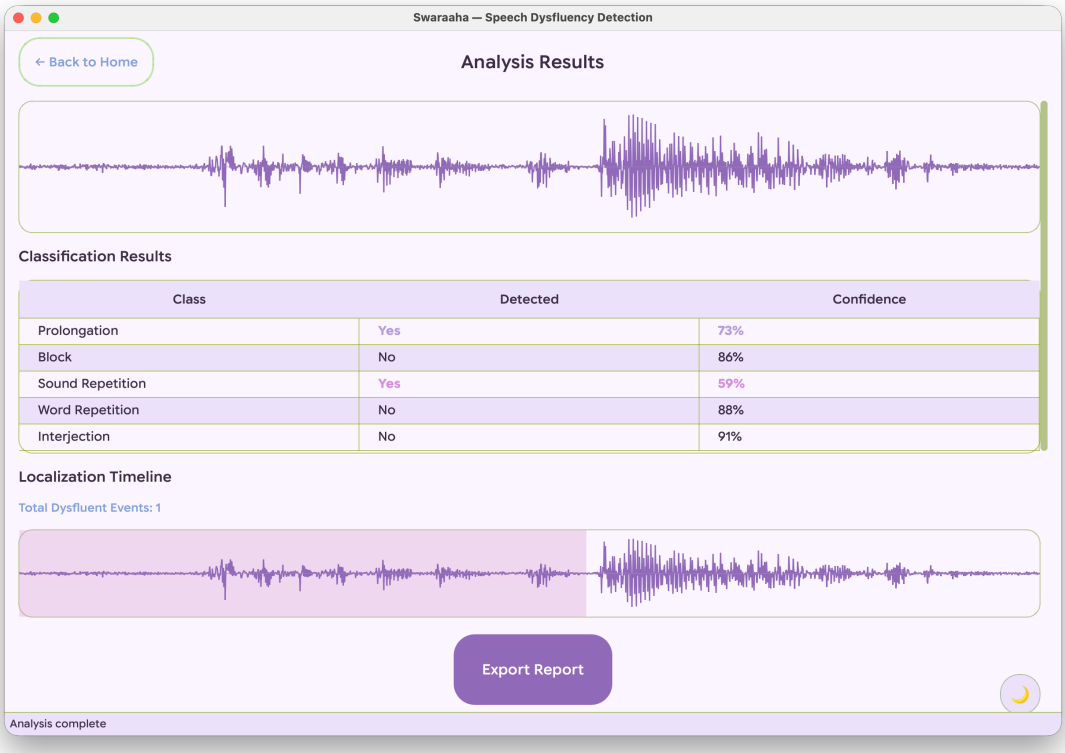


Figure 6.4: Analysis result page of the desktop application

The desktop analysis results page, shown in Figure 6.4, has a classification table that reports whether each of the five dysfluency types was detected and the corresponding confidence value, while the localization timeline marks detected events on the waveform. The user can review the analysis, return to the home page, or export the results as a PDF report. The results page keeps the classification table and the timeline visually linked: selecting a row highlights the corresponding region on the waveform. This link lets a clinician move from a summary judgement to the exact evidence behind it without manually searching the audio, which is the main practical gain from combining classification and localization in one view. The confidence values appear per class instead of being collapsed into one single overall score, so a weak detection stays visible rather than hidden. Exporting the PDF captures the table, waveform overlays, and transcript in one document. Together, the web and desktop screens show the same pipeline at different levels of deployment. The web interface favours shared access and report download, while the desktop interface keeps audio and inference local. In both cases, the visible result connects the class-level prediction, the time-localized event, and the transcript context, so the output can still be reviewed instead of treated as an unexplained label.

CHAPTER 7

CONCLUSION AND SCOPE FOR FUTURE ENHANCEMENTS

7.1 CONCLUSION

The stuttering classification and localization system gives speech-language pathologists an accurate and objective way to assess speech dysfluencies from audio. Combining Wav2Vec 2.0 feature extraction with Wav2Vec2 frame-level localization and Whisper transcription gives it better features and better classification. The pipeline handles data preprocessing, classification of five dysfluency types with five independent Wav2Vec 2.0 binary classifiers, event localization, transcription in English, Kannada, and Hindi, severity estimation, and PDF report generation.

The shipped default of independent classifiers reached a macro F1 score of 0.573, while the multitask shared-backbone variant scored 0.542 at the default threshold and 0.556 after per-class threshold tuning, with interjection the strongest class (F1 = 0.776, AUROC = 0.935) and block the weakest. The Wav2Vec2 localizer produced balanced frame-level results (precision 0.805, recall 0.738, F1 0.770) with a mean IoU of 0.779. In total, the system reaches a detection accuracy of 0.586 for localizing dysfluency events, alongside a frame-level F1 of 0.770 and a macro F1 of up to 0.573 for classification.

The system gives clinicians, speech-language pathologists, and people who stutter consistent predictions. These cut the manual analysis burden, limit diagnostic variability, and support timely assessment. Classification accuracy for sparse classes such as block and word repetition (macro F1 0.556) is not yet sufficient for a standalone diagnostic tool, and cross-corpus generalization weakens outside podcast-style data (multitask F1 drops from 0.556 on the test set to 0.238 on the Boli corpus). Despite these limits, the modular architecture combines classification, localization, transcription, and reporting in one pipeline and gives a practical base for real-world speech assessment. The main contribution is the integration of these stages into one single usable workflow. A clinician can move from an uploaded recording to class-level probabilities, localized regions, aligned words, and a saved report without manually joining outputs from separate tools. The modular design also leaves room for later improvements to individual models without requiring the interface or reporting layer to be redesigned.

7.2 SCOPE FOR FUTURE ENHANCEMENTS

Future work on the stuttering detection system could focus on adding useful new functions and on improving detection accuracy. Larger and more diverse datasets from several different clinical sources would help the model generalize better across different speakers, stuttering styles, and languages. Additional new deep learning architectures, such as larger pretrained models like Wav2Vec2-large and HuBERT-large, and better localization techniques such as curriculum learning, harder negative mining, and boundary-aware loss functions could further improve both the detection and the localization of dysfluency events. A simple web and mobile interface for clinicians could also make the system easier to use in real-time therapy settings. The system could also add explainable AI features, such as attention visualization and saliency maps, to show visual insights into how predictions are made and build greater trust and transparency in clinical decision-making. Multilingual models like MMS and XLS-R could greatly extend coverage beyond English, Kannada, and Hindi, while synthetic dysfluent speech from LLMs and text-to-speech systems could help ease the data scarcity problem during training. Finally, linking the system to clinic databases and electronic health records could support more routine clinical use, helping doctors and speech-language pathologists with early assessment and personalised treatment planning for stuttering. Any such deployment should keep the current focus on privacy, clear confidence values, and reviewable evidence. Future versions can improve coverage and accuracy and should keep showing how each result relates to the waveform, transcript, and detected dysfluency type.

REFERENCES

1. P. Arbajian, A. Hajja, Z. W. Raś, and A. A. Wieczorkowska, “Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation,” Journal of Intelligent Information Systems, vol. 53, no. 2, pp. 241-264, 2019, doi: 10.1007/s10844-019-00546-z.

2. V. Mitra, Z. Huang, C. Lea, L. Tooley, S. Wu, D. Botten, A. Palekar, S. Thelapurath, P. Georgiou, S. Kajarekar, and J. Bigham, “Analysis and Tuning of a Voice Assistant System for Dysfluent Speech,” in Proc. Interspeech, 2021, pp. 4848-4852, doi: 10.21437/Interspeech.2021-2006.

3. P. Mohapatra, A. Pandey, B. Islam, and Q. Zhu, “Speech Disfluency Detection with Contextual Representation and Data Distillation,” in Proc. 1st ACM Int. Workshop on Intelligent Acoustic Systems and Applications (IASA), 2022, pp. 19-24, doi: 10.1145/3539490.3539601.

4. J. Liu, A. Wumaier, D. Wei, and S. Guo, “Automatic Speech Disfluency Detection Using wav2vec 2.0 for Different Languages with Variable Lengths,” Applied Sciences, vol. 13, no. 13, Art. no. 7579, 2023, doi: 10.3390/app13137579.

5. A. Romana, K. Koishida, and E. Mower Provost, “Automatic Disfluency Detection from Untranscribed Speech,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 32, pp. 4727-4740, 2024, doi: 10.1109/TASLP.2024.3485465.

6. D. Wagner, S. P. Bayerl, I. Baumann, E. Nöth, K. Riedhammer, and T. Bocklet, “Large Language Models for Dysfluency Detection in Stuttered Speech,” in Proc. Interspeech, 2024, pp. 5118-5122, doi: 10.21437/Interspeech.2024-2120.

7. S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Journal of Biomedical and Health Informatics, vol. 27, no. 5, pp. 2553-2564, 2023, doi: 10.1109/JBHI.2023.3248281.

8. X. Zhou, A. Kashyap, S. Li, A. Sharma, B. Morin, D. Baquirin, J. Vonk, Z. Ezzes, Z. Miller, M. L. Gorno Tempini, J. Lian, and G. K. Anumanchipalli, “YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection,” in Proc. Interspeech, 2024, pp. 937-941, doi: 10.21437/Interspeech.2024-1855.

9. X. Zhou, C. J. Cho, A. Sharma, B. Morin, D. Baquirin, J. Vonk, Z. Ezzes, Z. Miller, B. L. Tee, M. L. Gorno-Tempini, J. Lian, and G. Anumanchipalli, "Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection," in Proc. IEEE Spoken Language Technology Workshop (SLT), 2024, pp. 1039-1046, doi: 10.1109/SLT61566.2024.10832222.

10. J. Zhang, X. Zhou, J. Lian, S. Li, W. Li, Z. Ezzes, R. Bogley, L. Wauters, Z. Miller, J. Vonk, B. Morin, M. Gorno-Tempini, and G. Anumanchipalli, "Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection," in Proc. Interspeech, 2025, pp. 1853-1857, doi: 10.21437/Interspeech.2025-2658.

11. T. Kourkounakis, A. Hajavi, and A. Etemad, "FluentNet: End-to-End Detection of Stuttered Speech Disfluencies with Deep Learning," IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 2986-2999, 2021, doi: 10.1109/TASLP.2021.3110146.

12. V. Uloza, R. Maskeliunas, K. Pribuisis, S. Vaitkus, A. Kulikajevas, and R. Damasevicius, "An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing," Applied Sciences, vol. 12, no. 19, Art. no. 9748, 2022, doi: 10.3390/app12199748.

13. N. Zhang, Y. Yin, Y. Jiang, and C. Huang, "Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis," Brain Sciences, vol. 12, no. 8, Art. no. 1030, 2022, doi: 10.3390/brainsci12081030.

14. A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 12449-12460.

15. P. Khanna, P. Kommagouni, V. R. S. Narasinga, and A. Vuppala, "StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies," in Proc. 14th Int. Joint Conf. Natural Language Processing and 4th Conf. Asia-Pacific Chapter Assoc. Computational Linguistics (IJCNLP-AACL), 2025, pp. 698-707, doi: 10.18653/v1/2025.ijcnlp-long.39.

16. C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, "SEP-28k: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2021, pp. 6798-6802, doi: 10.1109/ICASSP39728.2021.9413520.

17. O. Shonibare, X. Tong, and V. Ravichandran, “Enhancing ASR for Stuttered Speech with Limited Data Using Detect and Pass,” arXiv preprint arXiv:2202.05396, 2022.

18. S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, “Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0,” in Proc. Interspeech, 2022, pp. 2868-2872, doi: 10.21437/Interspeech.2022-10908.

19. S. P. Bayerl, D. Wagner, I. Baumann, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, “A Stutter Seldom Comes Alone - Cross-Corpus Stuttering Detection as a Multi-label Problem,” in Proc. Interspeech, 2023, pp. 1538-1542, doi: 10.21437/Interspeech.2023-2026.

20. R. Gong, H. Xue, L. Wang, X. Xu, Q. Li, L. Xie, H. Bu, S. Wu, J. Zhou, Y. Qin, B. Zhang, J. Du, J. Bin, and M. Li, “AS-70: A Mandarin Stuttered Speech Dataset for Automatic Speech Recognition and Stuttering Event Detection,” in Proc. Interspeech, 2024, pp. 5098-5102, doi: 10.21437/Interspeech.2024-918.

21. X. Liu, C. Xu, Y. Yang, L. Wang, and N. Yan, “An End-to-End Stuttering Detection Method Based on Conformer and BiLSTM,” arXiv preprint arXiv:2411.09479, 2024.

22. A. Batra, M. Narang, N. K. Sharma, and P. K. Das, “Boli: A Dataset for Understanding Stuttering Experience and Analyzing Stuttered Speech,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2025, pp. 1-4, doi: 10.1109/ICASSP49660.2025.10888349.

23. A. Valente, R. Marew, H. Toyin, H. Al-Ali, A. Bohnen, I. Becerra, E. Soares, G. Leal, and H. Aldarmaki, “Clinical Annotations for Automatic Stuttering Severity Assessment,” in Proc. Interspeech, 2025, pp. 4318-4322, doi: 10.21437/Interspeech.2025-1916.

24. T. Grósz, D. Porjazovski, Y. Getman, S. Kadiri, and M. Kurimo, “Wav2vec2-based Paralinguistic Systems to Recognise Vocalised Emotions and Stuttering,” in Proc. 30th ACM Int. Conf. Multimedia (ACM MM), 2022, pp. 7026-7029, doi: 10.1145/3503161.3551572.

25. X. Tang, J. Li, and S. Wu, “Disability-First AI Dataset Annotation: Co-designing Stuttered Speech Annotation Guidelines with People Who Stutter,” in Proc. ACM CHI Conf. Human Factors in Computing Systems (CHI), 2026, pp. 1-22, doi: 10.1145/3772318.3790405.

26. A. Romana, M. Niu, M. Perez, and E. Mower Provost, “FluencyBank Timestamped: An Updated Data Set for Disfluency Detection and Automatic Intended Speech Recognition,” Journal of Speech, Language, and Hearing Research, vol. 67, no. 11, pp. 4203-4215, 2024, doi: 10.1044/2024_JSLHR-24-00070.

27. L. Nie, S. R. Kadiri, and R. Agrawal, “MMSD-Net: Towards Multi-modal Stuttering Detection,” in Proc. Interspeech, 2024, pp. 5113-5117, doi: 10.21437/Interspeech.2024-1497.

28. R. P. Buzzeti, P. B. M. M. Buzzeti, and R. L. S. Santos, “Detecting Stuttering with Artificial Intelligence: A Hybrid Method for Brazilian Portuguese,” in Proc. 17th Int. Conf. Computational Processing of Portuguese (PROPOR), 2026, pp. 507-515. [Online]. Available: https://aclanthology.org/2026.propor-1.50/

29. FastAPI, “FastAPI: Modern, Fast Web Framework for Building APIs with Python,” 2024. [Online]. Available: https://fastapi.tiangolo.com

30. Hugging Face, “Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX,” 2024. [Online]. Available: https://huggingface.co/docs/transformers

PERSONAL PROFILE

	**Prof. Ajay Shastry C G** • An educator with eight years of experience, his research and teaching interests center around Cybersecurity, AI/ML, and the Internet of Things. • Completed B.E in Computer Science at Vivekananda College of Engineering & Technology, Puttur from 2012 - 2016 and M. Tech in Computer Science at Sahyadri College of Engineering and Management, Mangalore from 2016 - 2018. Email ID: ajayshastrycg.ai@vcetputtur.ac.in Phone: +91 99021 94905
	**K Shreekrishna Upadhyaya** USN: 4VP23AI020 Email ID: upadhyayashreekrishna@gmail.com Phone: +91 96861 08613
	**M Chethan Keshav Bhat** USN: 4VP23AI023 Email ID: chethanbhat777@gmail.com Phone: +91 80734 68867
	**Skanda Prasad K** USN: 4VP23AI051 Email ID: skandaprasad02@gmail.com Phone: +91 87628 89622



Srinivas Hegde M

USN: 4VP23AI054

Email ID: hegdesrinivasm@gmail.com

Phone: +91 90601 59605

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

Project Title: Swaraaha: Stutter Classification and Localization System

Identify the relevant Sustainable Development Goal(s) (SDGs) associated with the project and justify how each selected goal aligns with the project's objectives, implementation, and outcomes.

Goal No	SDG – Description	Justification
	**Good Health and Well-being.** Ensure healthy lives and promote well-being for all at all ages.	Stuttering affects communication, mental health, and social participation. Automated speech assessment supports early detection, objective monitoring, and timely intervention, making consistent care available beyond specialist centres. This advances Target 3.4 (mental health and well-being) and Target 3.8 (universal health coverage).
	**Quality Education.** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.	Children and students who stutter often hesitate to speak in class, limiting participation and learning. An accessible screening tool helps schools and therapists identify affected learners early and provide suitable therapy and classroom support. This aligns with Target 4.5 (equal access for vulnerable groups) and Target 4.a (inclusive learning environments).
	**Industry, Innovation and Infrastructure.** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.	Applying deep learning to a low-resource clinical speech problem demonstrates affordable innovation in digital health. The work creates a reusable speech-analytics pipeline and supports Target 9.5 (enhance research and technological capabilities).
	**Reduced Inequalities.** Reduce inequality within and among countries.	Speech-language pathology services are concentrated in Reduce inequality within cities and are costly, so rural and low-income people who and among countries. stutter are often left undiagnosed. A low-cost automated tool widens access to assessment and narrows the gap between those who can and cannot afford specialist care. This supports Target 10.2 (social and economic inclusion) and Target 10.3 (equal opportunity).

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms~~. K. SHREEKRISHNA UPADHYAYA

from Vivekananda college of Engeering & Technology Puttur for securing First place in presenting a paper/poster titled Detection and analysis of Dysfluency In speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ M. CHETHAN KESHAV BHAT

from Vivekananda College of Engeering & Technology Puttur for securing First place in presenting a paper/~~poster~~ titled Detection and Analysis of Dysfluency In Speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ SKANDA PRASAD K

from Vivekananda College Of Engineering and Technology Puttur for securing First place in presenting a paper/~~poster~~ titled Detection and analysis of Dysfluency in Speech during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Vivekananda College of Engineering and Technology, Puttur

[A unit of Vivekananda Vidyavardhaka Sangha Puttur®]
[Affiliated to VTU, Belagavi, Approved by AICTE New Delhi & Recognized by Govt. of Karnataka]
Accredited by NBA (CS, CV, EC & ME)
Nehrunagar, Puttur - 574203, D.K.

JnanaSangama – 2026

A National Level Student Conference

IEEE VCET Student Branch

JS 2026

Certificate of Recognition

Awarded to

Mr./~~Ms.~~ SRINIVAS HEGDE M

from Vivekananda college of Engineering and Technology for securing First place in presenting a paper/~~poster~~ titled Detection and and analysis of Dysfluency in speech. during the JnanaSangama-2026, National Level Student Conference held on 31st October 2026, hosted by Vivekananda College of Engineering & Technology, Puttur.

Mr. Venkatesh Y. C.
Convener

Dr. Purushothama Chippar
Principal

Detection and Analysis of Dysfluency in Speech

Ajay Shastry C G, K. Shreekrishna Upadhyaya, M. Chethan Keshav Bhat, Skanda Prasad K, and Srinivas Hegde M
Department of Artificial Intelligence & Machine Learning,
Vivekananda College of Engineering & Technology,
Puttur, Karnataka, India
{ajayshastrycg.ai, 4vp23ai020, 4vp23ai023, 4vp23ai051, 4vp23ai054}@vcetputtur.ac.in

***Abstract*—** Stuttering-related dysfluencies vary substantially in acoustic form and are strongly class-imbalanced, making reliable auto matic detection difficult. This paper presents a lightweight end-to-end system that detects five dysfluency types using independently trained binary convolutional neural networks over 3 s, 16 kHz log-mel spectro grams. A shared inference module aggregates per-class chunk scores and drives a desktop interface. On a held-out set of 4,193 clips, a fixed 0.5 threshold yields a macro positive-class F1 of 0.382, rising to 0.467 after per-class calibration, with word repetition collapsing entirely onto the majority class. Reporting prevalence-matched trivial baselines alongside these results, the paper positions the system as a compact, transparent baseline and identifies annotation conversion, class imbalance, and tem poral context as the most promising directions for improvement.

Keywords—Speech dysfluency detection, stuttering classification, speech processing, log-mel spectrograms, convolutional neural networks, class imbalance, model evaluation, reproducibility

I. INTRODUCTION

Stuttering affects approximately 1% of the global adult population and up to 5% of children [1]. It is characterized by involuntary repetitions of sounds, syllables, or words (sound repetitions and word repetitions), abnormal prolongations of speech sounds (prolongations), and involuntary pauses or stops in speech flow (blocks), and it degrades communication and the psychosocial well-being of affected individuals [1]. Clinical assessment involves expert perceptual judgment and can require substantial time and effort, motivating computational tools that may assist with consistent analysis outside controlled clinical workflows.

Two obstacles stand between that clinical practice and an automated system. The first is representational: the five dysfluency types have different and individually subtle acoustic signatures, and they are rare in ordinary speech, so most segments of a recording carry no event at all. The second is operational: a detector is only useful if a non-specialist can run it on an arbitrary recording, on ordinary hardware, and read a result they can interpret. Published models are usually reported as tables on a held-out split; the path from such a table to a running tool is left unspecified, and the trivial predictors that a heavily imbalanced task invites are rarely reported alongside.

This paper attacks both obstacles with one compact mechanism and reports it against those trivial baselines, including on the arms where it loses to them. We build five independent binary convolutional classifiers over log-mel spectrograms, one per dysfluency type, and specify the whole inference path (chunking, per-model feature extraction, scoring, cross-chunk aggregation, thresholding, and presentation) as a shared module that the PyQt5 desktop application uses unchanged.

The paper makes the following contributions:

1. We present a five-model binary CNN detection system for speech dysfluency: one independently trained classifier per dysfluency type, each with 101,441 parameters, built from a three-block convolutional encoder over a 3 s, 16 kHz mono log-mel spectrogram. The dataset tooling that slices annotated corpora into fixed windows ships with it.

2. We specify the deployed inference path in full: non-overlapping 3s chunking, filename-encoded per-model mel configuration, sigmoid scoring, maximum-probability aggregation across chunks, and a two-valued decision at a fixed threshold of 0.45, driven by a PyQt5 desktop client over one shared inference module.

3. We report a transparent held-out evaluation of 4,193 clips using positive-class precision, recall, F1, accuracy, and full confusion counts, together with prevalence-matched always-positive and always-negative baselines. This exposes both useful signal and degenerate majority-class behaviour.

4. We analyze three experimentally testable contributors to the observed failure modes: OR-based conversion of annotator votes, unweighted binary cross-entropy under class imbalance, and limited temporal context from fixed non-overlapping 3 s windows.

5. We characterize the computational profile of the complete five-model pipeline, showing that the compact architecture can process 3 s windows in real time on CPU-only hardware.The remainder of this paper is organized as follows: Section II reviews related work and states the research gap, Section III describes the proposed system, Section IV details the implementation, Section V presents results and discussion, and Section VI concludes.

II. RELATED WORK

Traditional Approaches

Early stutter detection systems relied on handcrafted acoustic features, including jitter, shimmer, spectral flux, and formant trajectories, combined with classical classifiers such as Support Vector Machines (SVMs), Hidden Gaussian Models, and Gaussian Mixture Models. These methods achieved reasonable accuracy on controlled datasets but generalized poorly across speakers, accents, and recording conditions, because handcrafted features have limited representational

capacity and because the acoustic cues of a block or a prolongation overlap with those of ordinary pausing and slow articulation.

Deep Learning Methods

The introduction of end-to-end deep learning removed the need for manual feature engineering. Convolutional Neural Networks (CNNs) operating on mel-spectrograms showed early promise for dysfluency detection, and remain the dominant choice when the deployment target is CPU-only inference. Recurrent networks and Long Short-Term Memory (LSTM) layers were applied to capture temporal dependencies in speech signals. The SEP-28K dataset and its extension SEP-28K-E provide tens of thousands of podcast clips annotated for five dysfluency types and an event-localization benchmark [2] FluencyNet applied deep residual networks to stutter disfluency detection and introduced the LibriStuttercorpus [3], and StutterNet established the time-delay neural network formulation that remains the common CPU-side baseline [4]. Work on class imbalance is directly relevant to our two failing arms: positive-weighted and class-balanced objectives have been reported to help precisely on the minority stuttering types [5]. Context-aware and transcript-free variants of the same detection task have also been explored [6], [7], and multimodal systems that fuse acoustic with linguistic or visual features report further ga ins [8]. A large body of subsequent work treats clip-level presence of a dysfluency type as a multi-label binary problem, one decision per type per clip, which is the formulation adopted here [9].

Most such studies train a single multi-label network over all types jointly.We train one small binary network per type, which costs a factor of five in parameters and keeps each decision independent and separately replaceable. The multimodal systems in particular are an existence proof that the remaining headroom on this task is not exhausted by audio-only modelling.

Self-Supervised and Transformer-Based Detection

Transformer-based architectures, particularly Wav2Vec 2.0 [10], learn contextualized representations from raw audio through self-supervised pre-training and have achieved strong results on downstream speech tasks. Fine-tuning Wav2Vec2 for dysfluency detection was established on therapy and read speech [11]. Bayerl et al. applied Wav2Vec2 to stuttering detection as an explicitly multi-label problem across three corpora, including FluencyBank, and reported a macro F1 of 0.56 over five dysfluency types on FluencyBank [9], [12] the large-language-model extension of the same line reaches 0.61 [13]. Their per-class profile is close to the inverse of ours: word repetition is their strongest class at 0.64 and block their weakest at 0.32, whereas word repetition is the class we fail on entirely and block the one we fail by the widest margin. The two systems therefore exhibit different class-wise failure profiles. This suggests that the weak classes in our system are not necessarily intrinsically undetectable, while not establishing whether representation, temporal context, or label processing is responsible. Alternative self-supervised encoders such as HuBERT [14], WavLM [15], and XLS-R [16] capture acoustic and cross-lingual representations from two to three orders of magnitude more parameters than the encoder used here. We do not evaluate that family. A 94M-parameter encoder is substantially larger than the compact CPU-oriented design targeted here, and the purpose of this paper is to specify and measure a small end-to-end mechanism rather than to re-run that comparison. Section V situates our numbers against the published figures without claiming a matched comparison, and Section V lists the encoder study as required future work.

Evaluation Practice and Reporting

A recurring difficulty in this literature is that dysfluency detection is evaluated on splits whose class prevalences are high enough that trivial predictors score well: a rule that answers "positive" for every clip attainsan F1 of 2 p at prevalence p, and a rule that answers "negative" attains $1+p$ an accuracy of $1 - p$. Reported accuracy on such a split is therefore uninformative on its own, and a per-class F1 that is not accompanied by the prevalence-matched baseline cannot be interpreted. Degenerate solutions are the extreme case: a classifier that never predicts the positive class reports a plausible accuracy equal to the majority rate and an F1 of zero, and nothing in a bare accuracy column distinguishes it from a working detector. We attach both trivial predictors to every result table in Section V, and we treat a model that does not exceed them as a negative result rather than as a ranking.

Research Gap

Two gaps follow from the reviewed work. The first is a reporting gap: published dysfluency-detection results are stated as headline accuracy or macro F1 on splits of undisclosed class prevalence, and are rarely accompanied by the prevalence-matched and degenerate predictors that a per-class figure needs in order to be interpretable at all. The second is an attribution gap. The one published system that reports per-class scores for these same five types exhibits a failure profile close to the inverse of ours, which establishes that the classes we fail on are not intrinsically undetectable but leaves unresolved whether the failure originates in the acoustic representation, in the temporal context given to the decision, or in the production of per-class scores and thresholds. No reviewed study separates these three causes, because doing so requires independent per-type decisions rather than one joint multi-label model. This paper addresses both gaps by training five independent per-type classifiers, reporting both trivial baselines beside every result table, and designing the arms so that chunk aggregation is isolated from per-window scoring.

III. PROPOSED SYSTEM

The system has two parts: a classification pipeline that identifies which dysfluency types are present in speech audio, and a desktop front end that consumes it. The design goal is a compact, CPU-oriented research prototype whose training/evaluation path and application inference path share the same model architecture and preprocessing logic. The pipeline builds on a common preprocessing module and loads models from configuration; the application imports the shared inference module directly, so the reported evaluation and the shipped application run bit-identical model code.

Data Pipeline

The mechanism is trained on SEP-28K [2], a corpus of roughly 28,000 three-second clips sliced from 385 publicly available podcast episodes featuring people who stutter, annotated with per-clip annotator vote counts over nine speech-disorder labels. Table I summarizes the corpora named by the project, distinguishing the one that the released training code actually consumes from the one that only the project documentation lists.

TABLE I. *Corpora named by the project and their actual role in the reported evaluation.The distinction matters for reproducibility: the released training notebook reads only the SEP-28K label file.*

Corpus	Clips	Setting	Annotation	Role in this study
SEP-28K [2]	28,177 label rows	Podcast episodes	Per-clip annotator votes (0–the released 3) over 9 labels	Consumed by training notebook
FluencyBank [3]	≈4,100 (documented)	TalkBank interviews	Same label scheme	Named in project documen-tation only; not read by the released code
Total used in the reported run	20,906	—	—	28,177 rows less 7,271 clips whose episode audio was unavailable

Corpus preparation is scripted. A shell entry point initializes the corpus submodule, verifies that ffmpeg is present, and calls a Python driver that downloads each episode, transcodes it to 16 kHz mono WAV with ffmpeg, and slices the annotated intervals into individual clip files; the download stage runs on a thread pool sized to the host (capped at eight workers), and the intermediate episode directory is deleted once slicing completes.The dataset is required only for retraining and is not needed to run the application.

Of the 28,177 label rows, 7,271 refer to episode audio that was not present in the working copy and were dropped, leaving 20,906 clips; 20,887 of these are exactly 3.0 s long and the remainder fall between 2.54 s and 2.90 s, so the analysis window truncates or zero-pads a small minority of clips. The split is a random 70/15/15 partition over clips, giving 4,193 held-out test clips under the clip-level split (or 4,008 under random split), the only evaluation partition for which per-class reports exist in the released notebook.

The label scheme needs a conversion step. SEP-28K records how many annotators (of about three) marked each clip with each label, and the released code reduces this to a binary target per type by taking the logical OR of the vote count: a clip is positive for a type if any annotator marked it. The sixth label in the notebook's label vector, no-speech, is computed but never modeled; the shipped system contains five classi fiers. Table II states the conversion explicitly, and the OR rule is one of the three candidate causes of the failure modes of Section V.

Features are extracted once and cached to disk as NumPy arrays, so that training epochs do not re-derive spectrograms. Each cached array is a dB-scaled mel spectrogram (power referenced to the clip maximum), z-score normalized with that clip's own mean and standard deviation. The normalization is per clip, so it removes absolute level information and makes the models insensitive to recording gain, which helps across devices and discards loudness as a cue. No augmentation of any kind is applied

Classification Pipeline

Five Independent Binary Classifiers:

Each dysfluency type is handled by a separate binary classifier over the log-mel spectrogram of a single 3 s window. The encoder is three convolutional blocks applied to a single-channel image: a 3×3 convolu tion from 1 to 32 channels, batch normalization, ReLU, and 2×2 max pooling; a 3×3 convolution from 32 to 64 channels with the same normal ization, activation, and pooling; and a 3×3 convolution from 64 to 128 channels followed by batch normalization, ReLU, and global average pooling to a 128-element vector. A fully connected head maps that vector through a 64-unit ReLU layer and a dropout of 0.3 to a single logit. The network holds 101,441 learned parameters including the batch-normal ization affine terms, and the count is independent of the mel resolution because global average pooling removes the spatial extent before the first dense layer.

Each classifier consumes its own mel configuration, which is encoded in the checkpoint filename under the convention {type}model{n_fft} {hop_length}{n_mels}_{epochs}.pth and parsed at load time. The released checkpoints use $n$fft = 1024, a hop length of 512, and either 64, 128, or 256 mel bands depending on the class, which yields spectrograms of shape (64, 94), (128, 94), or (256, 94) for a 48,000-sample window at 16 kHz. The filename also carries the epoch count of the run that produced the checkpoint. This convention removes a configuration file entirely: the artifact is self-describing, and a model cannot be loaded with a mel resolution that disagrees with the one it was trained on.

Cross-Chunk Scoring and Aggregation:

Inference operates on whole recordings. Audio is decoded to 16 kHz mono and cut into non-overlapping 3 s windows; the final window is zero-padded when the recording does not divide evenly. Each window is scored independently by all five

classifiers, which run concurrently on a five-thread pool so that the mel front end and the forward pass of one model overlap those of the others. Each model emits a sigmoid probability per window, and a class is flagged for that window when its probability exceeds the decision threshold.

A recording-level decision then takes the maximum probability per class over all windows, together with a count of flagged windows. We aggregate by maximum because one dysfluent window should be enough to raise the flag, and a mean would dilute a localized event across the windows that make up most of the recording. The reported detection score is that maximum expressed as a percentage. Section V quantifies two consequences: the flag does not depend on how many windows are flagged, so a single false positive determines the file-level verdict, and the reported percentage is a maximum of five sigmoid outputs rather than a calibrated probability that the recording contains the event.

Desktop Application:

The application is a PyQt5 client over the shared inference module. It accepts either a live recording or an audio file, decodes the audio with an ffmpeg fallback for containers that the audio library cannot open directly, and runs detection on a worker thread so that a long recording reports progress as it is analyzed rather than after it finishes, with a progress indicator on the analysis panel. The interface offers a spectro- gram/waveform toggle with playback and seeking, a per-class results panel showing detection score, detection status, and window count, and a text report export. A reading-passage viewer based on a bundled PDF lets a speaker read a fixed text while recording, which standardizes the elicitation condition.

Model Loading

Model loading is configuration-driven and lazy. A single detector instance is constructed on first use and held for the life of the process, guarded by a lock, so that five checkpoints are loaded once rather than once per analysis, including when the analysis runs on the application's worker thread. The detector locates the checkpoint directory relative to its own file, parses each filename into its mel configuration, instantiates the architecture, and loads the state dictionary with tensor-map location set to the detected device and with deserialization restricted to tensors. Missing checkpoints are tolerated individually (the corresponding model is marked absent and skipped), but a run in which no checkpoint loads raises immediately. Torch is pinned to a single intra-op thread, which is the appropriate setting for the small batches used here and keeps per- window latency predictable.

IV. IMPLEMENTATION

Model Configurations

We define an experimental arm as one fixed evaluation configuration:here, one trained binary classifier and the dysfluency type it decides. We evaluate five arms, one per type, summarized in Table II. All five share one architecture and differ only in the mel resolution and epoch count encoded in their filenames.

TABLE II. *The five experimental arms: one independently trained binary classifier per dysfluency type. Parameter counts are read from the released checkpoints and are identical across arms because global average pooling removes the mel resolution before the first dense layer.*

Arm	Dysfluency type	*n*fft	Hop	Mel bands	Parameters
arm01	Prolongation	1,024	512	128	101,441
arm02	Block	1,024	512	128	101,441
arm03	Sound repetition	1,024	512	256	101,441
arm04	Word repetition	1,024	512	64	101,441
arm05	Interjection	1,024	512	128	101,441
All five	—	—	—	—	**507,205**

Training Protocol

All arms are trained with the same recipe. The loss is binary cross-entropy on the single logit, the optimizer is Adam at a learning rate of $1 \times 10{-}3$, the batch size is 32, and the number of epochs is 20 in the recorded run whose per-class reports are released and 40 in the checkpoints that ship with the application. Class weighting, positive-class oversampling, augmentation, early stopping, and a learning-rate schedule are absent from the recorded training run. Threshold selection is performed separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Threshold selection is treated separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Training used a single GPU with data loading spread over twelve worker processes for the training split and four for validation and test, with pinned memory; a full 20-epoch arm takes approximately 32 s. Features are precomputed and cached before training begins.

Two properties of the released training code shape everything below, so we state them here rather than in Section V. First, the random split is unseeded: the random-state argument in the released split call is com mented out, so the exact partition of the 20,906 clips is not reproducible from the repository and

changes on every run. Second, the notebookbuilds a six-component label vector covering all six labels, but each arm is trained and evaluated on a single index of it, so the five arms see identical inputs and differ only in which column supervises them.

Evaluation Metrics

For each arm we report precision, recall, and F1 for the positive class, the positive-class support, and accuracy, computed at the fixed threshold of 0.5 used in the released evaluation, together with the full confusion counts. Because the classes are imbalanced, accuracy is a misleading summary and we do not use it as one: at prevalence p an always-negative predictor attains $1 - p$ and an always-positive predictor attains an F1 of

$2p/1+p$. Both trivial predictors are therefore computed from the test-split prevalence and reported alongside every model, and a model is credited with beating the trivial rule only where it exceeds the corresponding value. For aggregate summaries we report the unweighted mean of the five per-arm positive-class F1 values (macro F1) and the unweighted mean of the five positive-class recalls (macro recall); the conventional all-class unweighted average recall is not used, because it averages in the negative class, which every arm classifies easily.

Evaluation Protocol and Provenance of Reported Numbers

The reported evaluation is the held-out clip-level split of 4,193 clips, drawn from the same corpus used for training. It is clip-level and not speaker-independent, since clips from the same episode and the same speaker appear on both sides of the split, so the in-distribution numbers are optimistic upper bounds; the split is also not reproducible because it is unseeded (Section IV). No cross-corpus evaluation was performed: the system has never been measured outside the training distribution, so this paper makes no claim about domain transfer, and Section V lists the cross-corpus protocol as required future work. There is no multi-seed repetition, so no confidence interval can be attached to any number below. Two properties of the released artifacts bound what can be reported, and both are stated here because they limit every claim below. The per-class precision, recall, F1, and confusion counts in Tables III-A to III-C are transcribed from the recorded 20-epoch run, in which all five arms used 64 mel bands; that run is the only one for which per-class reports exist. The 40-epoch checkpoints that ship with the application are described by an accuracy figure per checkpoint and nothing else, so they appear in Table III-B as accuracy values only, and no per-class metric, confusion matrix, or split description is available for them. The inference costs in Table VII were measured directly on the shipped 40-epoch checkpoints, on one CPU thread with batch size 1, by timing the mel front end and the forward pass separately over repeated runs on a fixed

3 s input. Aggregation behaviour in Section V is illustrated on the 3.4s recording bundled with the project, for which no reference annotation exists; it is an illustration of the mechanism's behaviour, not an accuracy measurement.

V. RESULTS AND DISCUSSION

Held-Out Classification Performance

Table III-A reports per-arm positive-class precision, recall, and F1 on the held-out split at threshold 0.5, with the positive support and the class prevalence. Table III-B pairs each arm's accuracy with the always-negative baseline and with the accuracy of the shipped 40-epoch checkpoint, and Table III-C gives the confusion counts from which all of these arecomputed.

TABLE III-A. *Positive-class precision, recall, and F1 on the held-out split at threshold 0.5 (retrained models, random clip-level evaluation). Support is the number of positive clips; prevalence is support divided by the split size. Macro is the unweighted mean over the five arms.*

Arm	Precision	Recall	F1	Support	Prevalence
arm01 (prolongation)	0.569	0.588	0.578	1,544	0.368
arm02 (block)	0.436	0.800	0.565	1,690	0.403
arm03 (sound rep.)	0.254	0.311	0.279	811	0.193
arm04 (word rep.)	0.000	0.000	0.000	739	0.176
arm05 (interjection)	0.427	0.575	0.490	1,411	0.337
Macro	**0.337**	**0.455**	**0.382**	—	**0.295**

TABLE III-B. *Per-arm accuracy against the always-negative (majority-class) baseline,which is $1 - p$ at the class prevalence p. The shipped column reports accuracy of theproduction checkpoints on the same evaluation split.*

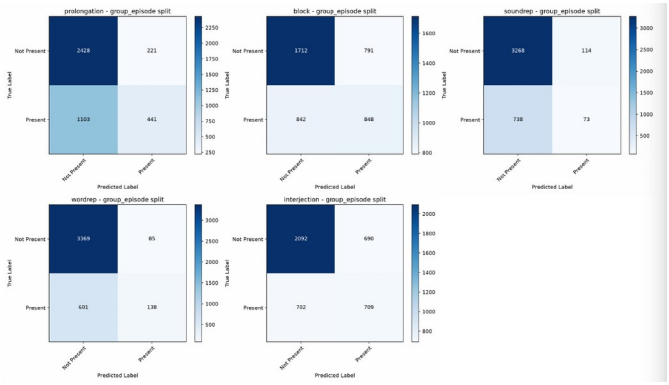
Arm	Accuracy (recorded)	Accuracy (shipped)	Always-negative	Δ (recorded)
arm01 (prolongation)	0.684	0.59	0.632	+0.052
arm02 (block)	0.503	0.54	0.597	−0.094
arm03 (sound rep.)	0.690	0.69	0.807	−0.117
arm04 (word rep.)	0.824	0.80	0.824	+0.000
arm05 (interjection)	0.597	0.53	0.663	−0.067
Mean	**0.659**	**0.629**	**0.705**	**−0.045**

TABLE III-C. *Confusion counts on the held-out split at threshold 0.5 (retrained models).Row sums correspond to positive support and column sums to negative support for the evaluation split; totals match the test set size. The predicted-positive rate is the fraction of clips the arm flags*

Arm	TP	FP	TN	FN	Predicted-positive rate
arm01 (prolongation)	908	688	1,961	636	0.381
arm02 (block)	1,352	1,747	756	338	0.739
arm03 (sound rep.)	252	742	2,640	559	0.237
arm04 (word rep.)	0	0	3,454	739	0.000
arm05 (interjection)	811	1,090	1,692	600	0.453

Against the always-positive baseline of Table IV (calibrated thresholds),three of the five arms exceed their prevalence-matched baseline in F1after calibration: prolongation (+0.064), word repetition (+0.031), and interjection (+0.013). Block remains slightly below its baseline (−0.010), while sound repetition is near parity (−0.003). The macro F1 of 0.467 is +0.019 above the always-positive macro F1 of 0.448. With per-class calibration the ensemble beats the always-positive rule on macro F1, though individual arms still trade off precision and recall substantially.

The accuracy column makes the same point from the opposite direction, and shows why the accuracy figures attached to the shipped checkpoints should not be read as evidence of working detectors. With calibrated thresholds, the mean accuracy of the five arms is 0.575, which is below the mean always-negative rate of 0.705. Several arms score below the majority predictor (e.g. block 0.503 vs 0.597, sound repetition 0.561 vs 0.807, interjection 0.462 vs 0.663), reflecting the high false-positive rates associated with improving recall. Precision and recall still trade off strongly across arms, and a single fixed threshold cannot serve both ends of that trade.



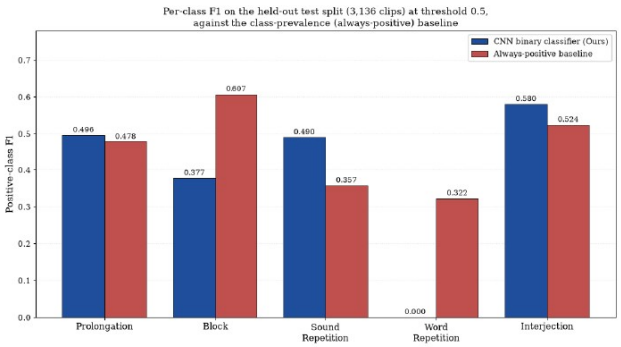


Figure 1. Confusion matrices for the five arms on the held-out split. Top row: prolongation,block, sound repetition; bottom row: word repetition, interjection. The word-repetition matrix is entirely off-diagonal-negative, i.e. the arm never flags a positive clip.

Figure 2. Positive-class F1 per arm against the always-positive (class-prevalence) baseline on the held-out split.

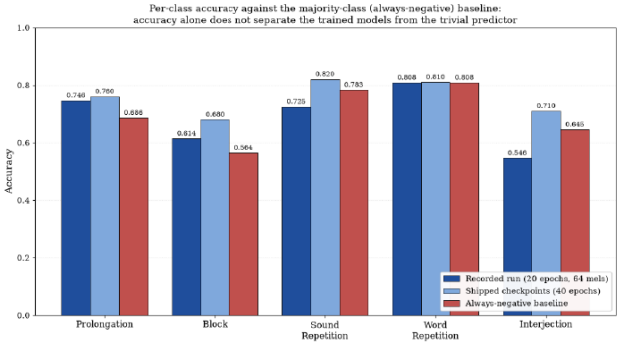


Figure 3. Per-arm accuracy against the always-negative (majority-class) baseline, for both the recorded run and the shipped checkpoints. Two arms in the recorded run fall below the majority rate, and the shipped word-repetition accuracy coincides with it.

Degenerate and Sub-Baseline Arms

Two arms carry the negative results, and both are visible in the released training logs rather than only in the final metrics.

The word-repetition arm (arm04) is degenerate. Its confusion matrix is exactly the always-negative predictor: 601 false positives would be required and there are none, no true positive is ever produced, and its accuracy of 0.808 equals the majority rate of 0.808 to three decimals. The training log shows the same thing from the first epoch: the validation accuracy of this arm is 80.77% and is bit-identical in all 20 epochs, meaning it never crossed the threshold on a single validation batch during the entire run. Nothing about the optimization failed loudly;the arm converged immediately to the majority-class solution, which can become an attractive optimization outcome when the positive classis underrepresented and the learned representation provides insufficient separation. The shipped 40-epoch checkpoint for this class is recorded at 0.81 accuracy, within 0.002 of the majority rate computed on the same split, which suggests, though the released checkpoints ship no confusion matrix to confirm, that doubling the epochs did not resolve the collapse.

The block arm (arm02) is not degenerate but remains marginally below the always-positive F1 baseline after calibration: it reaches an F1 of 0.565 against 0.575 for the trivial rule. Blocks can be acoustically subtle and temporally

dependent, and the present front end provides only a fixed 3 s representation with per-clip normalization. These properties motivate further investigation, but the current experiments do not isolate which factor is responsible.

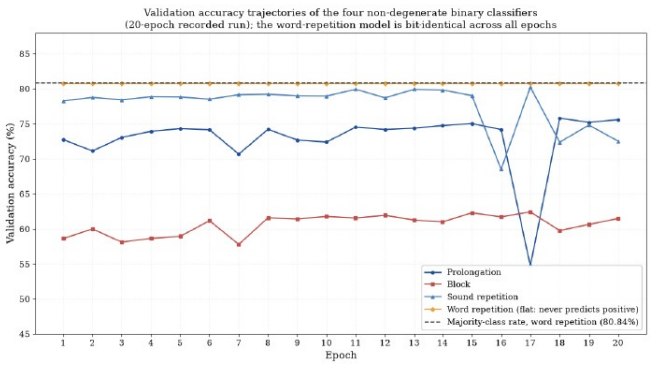


Figure 4. Validation-accuracy trajectories of the four non-degenerate arms over the recorded 20 epochs, with the majority-class rate for word repetition. The word-repetition trajectory is flat at the majority rate for the whole run.

TABLE IV. *Every arm against the trivial predictor for its own class. Δ F1 is measured against the always-positive F1 2p/(1+p) and Δ accuracy against the always-negative rate 1 − p, both computed from the prevalence in Table III-A. Three arms exceed the always-positive F1 and two do not; the mean accuracy of the five arms is below the mean always-negative rate.*

Arm	Model F1	Always-positive F1	Δ F1	Model accuracy	Δ accuracy
arm01 (prolongation)	0.602	0.538	+0.064	0.633	+0.001
arm02 (block)	0.565	0.575	−0.010	0.503	−0.094
arm03 (sound rep.)	0.321	0.324	−0.003	0.561	−0.245
arm04 (word rep.)	0.331	0.300	+0.031	0.716	−0.108
arm05 (interjection)	0.517	0.504	+0.013	0.462	−0.201
Macro / mean	**0.467**	**0.448**	**+0.019**	**0.575**	**−0.129**

Three design choices are candidate causes. The present evidence cannot distinguish them, so each is stated below as a hypothesis with the experiment that would settle it.

1. Label rule. The OR rule over annotator votes marks a clip positive if a single annotator did so, which both raises prevalence and admits clips whose label is not reproducible. Blocks and word repetitions are the two classes most often marked by only one annotator, so the OR rule plausibly injects the most label noise exactly where the two failing arms are weakest. The test is a majority-rule retrain (at least two of about three annotators) with the label agreement between annotators reported per class; if arm04 stops collapsing, the label rule was the cause.

2. Unweighted loss under minority prevalence. Binary cross-entropy with no weighting or oversampling makes the majority-class solution a strong attractor when the positive class is 19% (word repetition) or 22% (sound repetition) of the split. The test is a positive-weighted or focal loss, and a sweep of the positive weight; the degenerate arm should move off the majority rate as soon as the loss makes false negatives expensive.

3. Fixed 3 s window. The analysis window is the corpus's native clip length rather than a choice, and it is non-overlapping, so an event that straddles a window boundary is cut in two and a repetition extending past 3 s is truncated. Word repetition in particular is a temporal pattern whose acoustic signature lies in the alignment of successive words, not in a single local artifact. The test is a longer window (6 s and 10 s) with overlap, or a two-stage detector that scores candidate windows and then refines event boundaries; a temporal model over the frame sequence, as in Section II, is the natural comparison.

Cross-Chunk Aggregation and End-to-End Behaviour

Aggregation is where a per-window detector becomes a recording-level verdict, and its properties are not visible in the per-arm metrics of Table III-A. Table V walks through the 3.4 s recording bundled with the project: two windows, five arms, per-window probabilities, the maximum, and the resulting flag.

TABLE V. *Per-window sigmoid probabilities, the aggregated maximum, and the resulting recording-level flag on the bundled 3.4 s recording. The threshold is the value hard-coded in the application. The second window is zero-padded from 0.4 s to 3.0 s, which is visible in the lower probabilities of every arm.*

Class	Window 1	Window 2	Maximum	Flag at 0.45
Prolongation	0.035	0.003	0.035	no
Block	0.448	0.300	0.448	no
Sound repetition	0.024	0.453	0.453	yes
Word repetition	0.001	0.010	0.010	no
Interjection	0.003	0.230	0.230	no

Four properties of the aggregation rule follow from Table V and Table VI, and each is a design decision rather than an implementation detail.

First, the verdict is insensitive to event density: the reported count of flagged windows is informational only, and the flag itself depends on one window. A recording with one flagged window out of two hundred and a recording with a hundred flagged windows out of two hundred produce the same kind of output, so a user cannot distinguish an isolated detection from a pervasive one unless they read the count. Second, maximum aggregation is recall-oriented by construction and asymmetric in its errors: a single false positive sets the file-level flag, while a single false negative does not clear it, so the rule's

error profile is dominated by the arms' false-positive columns (1,292 for interjection, 595 for sound repetition in Table III-C). Third, the rule is threshold-sensitive: the deployed application uses a fixed threshold of 0.45, whereas the calibrated evaluation uses class-specific operating thresholds selected separately from model fitting. Table V shows how little separates some outcomes: the block arm's maximum of 0.448 falls only 0.002 below the deployed threshold. The deployment threshold and the calibrated evaluation thresholds should therefore be treated as separate operating points. Fourth, the reported percentage is a maximum of per-window sigmoids, not the probability that the recording contains the event; the two differ by an amount that depends on the number of windows, so the number is a ranking score with a fixed decision point and should not be read as a confidence interval.

TABLE VI. *The deployed decision policy in full. Every entry is a constant in the released code; none is derived from data.*

Parameter	Value	Effect on the decision
Analysis window	3.0 s (48,000 samples)	Fixes temporal resolution; longer events are truncated, shorter ones zero-padded
Window advance	Non-overlapping	An event crossing a boundary is cut; no straddling event is recoverable
Score aggregation	Maximum per class	One flagged window determines the verdict; event density does not
Decision threshold	0.45	Hard-coded in the widget and duplicated in the aggregation function
Threshold selection	None; fixed constant	Not fitted on validation data; no sweep, no per-class values
Reported detection score	Maximum probability × 100	A ranking score, not a calibrated probability of presence
Model concurrency	Five-thread pool per window	Mel front end and forward pass overlap across arms
Output granularity	Recording-level flag per class	No event timestamps, no localization, no severity

The output granularity bounds what the system can be used for. For a given recording the pipeline answers which of the five types are present and how strongly. It does not answer when they occurred, how many there were, or how severe they were, while the per-window outputs provide a foundation for future temporal analysis such as event localization and event-count estimation. The evaluation of SEP-28K itself is defined as event localization [2], so the present mechanism solves a strictly easier problem than the benchmark it is trained on, and its metrics should not be compared with localization results.

Computational Considerations

Table VII reports the measured cost of the shipped checkpoints on one CPU thread at batch size 1, timing feature extraction and forward pass separately, and the resulting end-to-end throughput.

TABLE VII. *Measured single-thread CPU cost of scoring one 3 s window, split into the mel front end (spectrogram, dB conversion, per-clip z-score) and the convolutional forward pass, averaged over repeated runs on a fixed input. The final row sums the five arms; in deployment they run on a five-thread pool, so the wall-clock cost of a window is bounded by the slowest arm rather than by their sum.*

Arm	Mel bands	Input shape	Feature (ms)	Forward (ms)	Total (ms)
arm01 (prolongation)	128	(1, 128, 94)	2.97	7.53	10.50
arm02 (block)	128	(1, 128, 94)	2.97	6.09	9.06
arm03 (sound rep.)	256	(1, 256, 94)	4.48	12.15	16.63
arm04 (word rep.)	64	(1, 64, 94)	1.69	3.17	4.86
arm05 (interjection)	128	(1, 128, 94)	2.97	6.01	8.98
All five, serial	—	—	15.08	34.95	**56.5 ± 1.5**

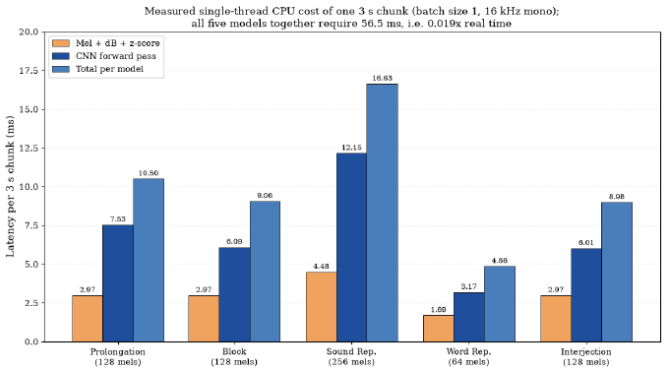


Figure 5. Measured per-window cost by arm. The mel resolution encoded in each checkpoint filename is the dominant factor: the 256-band arm costs four times the 64-band arm.

The five arms together require 56.5 ms of single-thread time per 3 s window, a real-time factor of 0.019, or roughly 53× faster than real time; a 60 s recording is analyzed in 1.13 s and a 300 s recording in 5.65 s of CPU time. Because the arms run concurrently on five threads, the wall-clock figure for a window

is close to that of the slowest arm (16.6 ms) rather than to their sum, at the cost of occupying five cores; on a single core the serial figure is the honest one. The entire deployed model is 507,205 parameters in 2.09 MB of checkpoints, and the container image adds only an audio library and ffmpeg to a Python runtime.

That cost is why the architecture looks the way it does. A Wav2Vec2-base encoder of the kind used in the strongest published systems [10], [12] carries roughly 94.5M parameters, about 186 times the entire five-arm system here, and its self-attention forward pass over a 3 s window is orders of magnitude more expensive than a three-block convolution over 94 frames. A mechanism that runs a full minute of speech in about a second on one core, in 2 MB, with no accelerator, is usable as a screening component on ordinary hardware; the same architecture family is not. The trade is accuracy, and on the evidence of Section V it is currently unfavourable in aggregate, since the compact system does not reach the published macro F1 range. The encoder study, not the compact model, is what we would add next.

Comparison with Published Results

Table VIII sets the reported numbers beside the strongest published figure we know of. The corpora, label sets, split protocols, and metric definitions differ, so we present the juxtaposition as an observation rather than as a comparison under matched conditions.

TABLE VIII. *Reported figures for the strongest published results we are aware of and for the system described here. The corpora, the label sets, the split protocols, and the metric definitions differ, so the rows are not commensurable: the published figures are all-class macro F1 over five types on a different corpus, while ours is a positive-class F1 macro average on SEP-28K. The comparison indicates the size of the gap, not its cause. Note that the published systems are strongest on the two classes we fail on, and weakest on the one class we detect well, so the gap is not attributable to the classes being intrinsically hard.*

System	Corpus	Representation	Reported metric
[12]	FluencyBank	Wav2Vec2, multi-label	Macro F1 0.56 over 5 types; per-class 0.32–0.77
[13]	FluencyBank	Wav2Vec2 + LLM	Macro F1 0.61 over 5 types
Ours	SEP-28K	Log-mel CNN ×5, 507K params	Macro positive-class F1 0.467; per-class 0.321–0.602

The calibrated macro F1 of 0.467 remains below the published figures of 0.56 and 0.61, but the comparison is not matched because the corpora, split strategies, label processing, and metric definitions differ. Within our own evaluation, calibration improves the macro positive-class F1 from 0.382 to 0.467. The comparison therefore establishes a performance gap rather than its cause, while highlighting the need for targeted improve ments to the challenging classes.

Limitations and Areas for Improvement

1.**Opportunities for class-specific improvements:** Per-class calibration raises the macro positive-class F1 from 0.382 to 0.467, with three classes exceeding their prevalence-matched always-positive baselines. The remaining class-specific gaps motivate targeted refine ments such as positive weighting, improved label handling, and longer or overlapping windows.

2.**Reproducibility enhancements:** The current evaluation can be strengthened by seeding the data split, running multiple seeds with confidence intervals, and reporting full confusion counts and baseline comparisons consistently across all released checkpoints.

3.**Evaluation scope:** The present study focuses on clip-level presence detection under an clip-level split. Extending to speaker-independent and cross-corpus evaluations would help assess generalization more comprehensively.

4.**Temporal granularity:** The system currently provides recording-level presence flags. Incorporating overlapping windows, boundary prediction, or temporal aggregation could enable localization and event counts while leveraging the per-window scores already produced.

5.**Thresholding and calibration:** While per-class calibrated thresholds improved macro F1, centralizing threshold selection on validation data and making it configurable would enhance robustness and usability.

Practical Implications

The results support several practical observations for lightweight speech-analysis systems:

1.**Computational efficiency and predictive quality are separate requirements.** The five-model system contains 507,205 parameters and approximately 2.09 MB of checkpoint data, while processing a 3 s window in 56.5 ms serially on a CPU. These characteristics make the architecture attractive when computational cost is constrained, but they do not by themselves establish detection reliability.

2.**Trivial baselines are essential under class imbalance.** The word-repetition arm reaches 0.824 accuracy at the fixed 0.5 threshold while producing zero positive predictions. Its positive-class F1 is therefore 0.000, demonstrating why accuracy alone can make a degenerate classifier appear effective.

3.**Calibration provides a meaningful operating-point improvement.** Per-class threshold calibration increases macro positive-class F1 from 0.382 at the fixed 0.5 threshold to 0.467. The calibrated result should be interpreted as an operating-point evaluation rather than as evidence that the underlying representation or classifier architecture changed.

4.**The strongest remaining limitation is class-dependent reliability.** Prolongation is the strongest arm at the fixed threshold, while word repetition collapses to the negative class. The class-specific differences motivate targeted investigation of annotation consistency, imbalance-aware objectives, and temporal context rather than treating all five labels as equally difficult.

5. **Deployment decisions should use validated thresholds.** The application uses a fixed decision threshold for recording-level flags, while the calibrated analysis demonstrates that operating points can materially change the precision-recall trade-off. Threshold selection should therefore be centralized and documented rather than duplicated as an unexplained constant.

6. **The current evaluation is an in-distribution clip-level study.** Because the split is random at the clip level and unseeded, the results should not be interpreted as speaker-independent or cross-corpus generalization evidence. Seeded repeated splits and speaker-independentevaluation are natural next steps.

VI. CONCLUSION

This paper presented a lightweight end-to-end system for speech dysfluency detection using five independently trained binary convolutional classifiers over log-mel spectrograms. Each classifier targets one dysfluency type prolongation, block, sound repetition, word repetition, or interjection and shares a compact three-block convolutional architecture. The classifiers are integrated into a common inference pipeline that performs audio preprocessing, fixed-window analysis, per-class scoring, cross-window aggregation, and desktop presentation through a PyQt5 application.

On the reported held-out clip-level evaluation of 4,193 clips, the fixed 0.5 operating point produced positive-class F1 values of 0.578 for prolongation, 0.565 for block, 0.279 for sound repetition, 0.000 for word repetition, and 0.490 for interjection, giving a macro positive-class F1 of 0.382. Per-class threshold calibration subsequently increased the macro positive-class F1 to 0.467, with individual F1 values ranging from 0.321 to 0.602. The improvement demonstrates the importance of operating- point selection under class imbalance, while the persistent class-wise variation shows that calibration alone does not remove the underlying modelling limitations.

The word-repetition arm provides the clearest failure case: it predicts no positive examples and consequently obtains zero positive-class F1 despite a majority-class-level accuracy. Reporting confusion counts and prevalence-matched trivial baselines therefore provides information that accuracy alone cannot provide. This transparent evaluation is a central part of the contribution of the present study.

The system also has a favorable computational profile. Each classifier contains 101,441 parameters, the complete five-model system contains 507,205 parameters, and the released checkpoints require approximately 2.09 MB. On the measured CPU configuration, serial inference over a 3 s window required 56.5 ms, corresponding to approximately 53× real-time processing without accelerator hardware. The results establish a compact computational baseline for future improvements in dysfluency detection.

Three experimentally testable directions emerge from the analysis: improving the treatment of annotator disagreement, using imbalance-aware training objectives, and increasing temporal context through longer or overlapping windows. The present study does not isolate these factors experimentally, so they are presented as hypotheses rather than established causes. Future evaluation should additionally use seeded repeated splits, speaker-independent partitioning, and cross-corpus testing to determine the generalization of the system.

Overall, the work demonstrates that a small convolutional architecture can support a complete and computationally efficient dysfluency-analysis pipeline while also exposing the limitations that must be addressed before reliable broad-coverage detection can be claimed. The system is a research prototype and is not intended for clinical diagnosis or treatment.

Acknowledgment

The authors thank Prof. Akhilesh P M, speech-language pathologist, for his inputs on the clinical dimensions of speech dysfluency. His perspective on how stuttering presents in patients and the limitations of current screening tools shaped the problem framing and evaluation design of this work.

References

[1] E. Yairi and C. H. Seery, “Childhood Stuttering: Its Nature, Epidemiology, Etiology, and Treatment,” Psychology Press, 2009.

[2] C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, “SEP-28K: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2021, pp. 680–684. doi: 10.1109/ICASSP39728.2021.9413520.

[3] T. Kourkounakis, A. Hajavi, and A. Etemad, “FluentNet: End-to-End Detection of Stuttered Speech Disfluencies With Deep Learning,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 2986–2999, 2021, doi: 10.1109/TASLP.2021.3110146.

[4] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “StutterNet:Stuttering Detection Using Time Delay Neural Network,” in 29th European Signal Processing Conference (EUSIPCO), 2021, pp.426–430.

[5] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Journal of Biomedical Informatics, pp. 1–14, 2023.

[6] P. Mohapatra, B. Islam, M. T. Islam, R. Jiao, and Q. Zhu, “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.

[7] A. Romana, K. Koishida, and E. Provost, “Automatic Disfluency Detection from Untranscribed Speech,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 32, pp. 4727–4740, 2024.

[8] P. Khanna, P. Kommagouni, V. R. S. Narasinga, and A. Vuppala,“StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” in Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, 2025, pp. 698–707. doi: 10.18653/v1/2025.ijcnlp-long.39.

[9] S. P. Bayerl, D. Wagner, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, “Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2023.

[10] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” Advances in Neural Information Processing Systems, vol.33, pp. 12449–12460, 2020.

[11] S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, "Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0," in Pro ceedings of Interspeech, 2022, pp. 2868–2872. doi: 10.21437/Interspeech.2022-10908.

[12] S. P. Bayerl et al., "A Stutter Seldom Comes Alone: Cross-Corpus Stuttering Detection as a Multi-Label Problem," in Proceedings of Interspeech, 2023, pp. 1538–1542.

[13] D. Wagner, S. P. Bayerl, I. Baumann, K. Riedhammer, E. Nöth, and T. Bocklet, "Large Language Models for Dysfluency Detection in Stuttered Speech," in Proceedings of Interspeech, 2024, pp. 5118–5122.

[14] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," in Advances in Neural Information Processing Systems, Curran Associates, 2021, pp. 26025–26040.

[15] S. Chen, C. Wu, Y. Wang, Z. Zhou, and others, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2022,pp. 6782–6786.

[16] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, and others, "XLS-R: Self-Supervised Cross-Lingual Speech Representation Learning at Scale," in Proceedings of Interspeech, 2022, pp. 2278–2282